

SOLIDWORKS API (NET VBA C#) — Examples (37/50): Insert_Boundary_Surface_Feature_Example_VB-

SOLIDWORKS API Documentation (converted from CHM)

Import DXF/DWG File Example (VBA)

This example shows how to import a DXF/DWG file.

```
'-----
' Preconditions: Verify that the specified DXF file exists.
'
' Postconditions:
' 1. Imports the specified file into SOLIDWORKS Design.
' 2. Examine the drawing.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim filename As String
    Dim boolstatus As Boolean
    Dim longerrors As Long

    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\importexport\rainbow.DXF"

    Set swApp = Application.SldWorks

    ' Get the specified DXF/DWG import data
    Dim importData As SldWorks.ImportDxfDwgData
    Set importData = swApp.GetImportFileData(filename)

    ' To let SOLIDWORKS Design determine an appropriate input file unit, do not set the LengthUnit property
    ' importData.LengthUnit("") = SwConst.swLengthUnit_e.swMETER
    ' importData.LengthUnit("") = SwConst.swLengthUnit_e.swFEET
    ' importData.LengthUnit("Model") = SwConst.swLengthUnit_e.swFEET
    ' importData.LengthUnit("Sheet2") = SwConst.swLengthUnit_e.swMETER

    ' To let SOLIDWORKS Design determine an appropriate output paper size, do not set the PaperSize values
    ' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperA3size, 0#, 0#)
    ' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperEsize, 0#, 0#)
    ' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPapersUserDefined, 0.5, 0.8)
    ' boolstatus = importData.SetPaperSize("Model", SwConst.swDwgPaperSizes_e.swDwgPaperA3size, 0.5, 0.8)
    ' boolstatus = importData.SetPaperSize("Sheet2", SwConst.swDwgPaperSizes_e.swDwgPapersUserDefined, 0.16, 0.14)

    ' To let SOLIDWORKS Design determine an appropriate sheet scale, do not set the SheetScale values
    ' boolstatus = importData.SetSheetScale("", 1#, 12#)
    ' boolstatus = importData.SetSheetScale("Model", 1#, 3#)
    ' boolstatus = importData.SetSheetScale("Sheet2", 1#, 1#)

    ' To let SOLIDWORKS Design determine an appropriate sheet name, do not set the SheetName property
    ' importData.SheetName("Model") = "S1"
    ' importData.SheetName("Sheet2") = "S2"

    ' Load the specified DXF/DWG file
    Dim newDoc As SldWorks.ModelDoc2
    Set newDoc = swApp.LoadFile4(filename, "", importData, longerrors)

End Sub
```

Import DXF File into Part Sketch Example (VBA)

This example shows how to import a DXF file to a part sketch.

```
'-----
' Preconditions:
' 1. Verify that the specified DXF file exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Imports the specified file into SOLIDWORKS Design.
' 2. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim filename As String
    Dim longerrors As Long
    Dim retVal As Boolean
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\importexport\rainbow.DXF"

    Set swApp = Application.SldWorks
    Dim importData As SldWorks.ImportDxfDwgData
    Set importData = swApp.GetImportFileData(filename)

    ' Import method
    importData.ImportMethod("") = SwConst.swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToPartSketch

    ' Load the specified DXF/DWG file
    Dim newDoc As SldWorks.ModelDoc2
    Set newDoc = swApp.LoadFile4(filename, "", importData, longerrors)

    ' Gets
    Debug.Print "Part Sketch Gets:"
    Debug.Print "  Add constraints:  " & importData.AddSketchConstraints("")
    Debug.Print "  Merge points:      " & importData.GetMergePoints("")
    Debug.Print "  Merge distance:    " & (importData.GetMergeDistance("") * 1000#) & " mm"
    Debug.Print "  Import dimensions: " & importData.ImportDimensions("")
    Debug.Print "  Import hatch:      " & importData.ImportHatch("")

    'Sets
    Debug.Print "Part Sketch Sets:"
    importData.AddSketchConstraints("") = True
    Debug.Print "  Add constraints:  " & importData.AddSketchConstraints("")
    retVal = importData.SetMergePoints("", True, 0.000002)
    Debug.Print "  Merge points:    " & retVal
    Debug.Print "  Merge distance:  " & (importData.GetMergeDistance("") * 1000#) & " mm"
    importData.ImportDimensions("") = True
    Debug.Print "  Import dimensions: " & importData.ImportDimensions("")
    importData.ImportHatch("") = False
    Debug.Print "  Import hatch:    " & importData.ImportHatch("")

End Sub
```

Import DXF File to Drawing Example (VBA)

This example shows how to import a DXF file to a drawing.

```
'-----
' Preconditions: Verify that the specified DXF file exists.
'
' Postconditions:
' 1. Imports the specified file into SOLIDWORKS.
' 2. Examine the drawing.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim filename As String
    Dim boolstatus As Boolean
    Dim longerrors As Long

    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\importexport\rainbow.DXF"

    Set swApp = Application.SldWorks

' Get the specified DXF/DWG import data
    Dim importData As SldWorks.ImportDxfDwgData
    Set importData = swApp.GetImportFileData(filename)

' To let SOLIDWORKS Design determine an appropriate input file unit, do not set the LengthUnit property
' importData.LengthUnit("") = SwConst.swLengthUnit_e.swMETER
' importData.LengthUnit("") = SwConst.swLengthUnit_e.swFEET
' importData.LengthUnit("Model") = SwConst.swLengthUnit_e.swFEET
' importData.LengthUnit("Sheet2") = SwConst.swLengthUnit_e.swMETER

' To let SOLIDWORKS Design determine an appropriate output paper size, do not set the PaperSize values
' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperA3size, 0#, 0#)
' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperEsize, 0#, 0#)
' boolstatus = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPapersUserDefined, 0.5, 0.8)
' boolstatus = importData.SetPaperSize("Model", SwConst.swDwgPaperSizes_e.swDwgPaperA3size, 0.5, 0.8)
' boolstatus = importData.SetPaperSize("Sheet2", SwConst.swDwgPaperSizes_e.swDwgPapersUserDefined, 0.16, 0.14)

' To let SOLIDWORKS Design determine an appropriate sheet scale, do not set the SheetScale values
' boolstatus = importData.SetSheetScale("", 1#, 12#)
' boolstatus = importData.SetSheetScale("Model", 1#, 3#)
' boolstatus = importData.SetSheetScale("Sheet2", 1#, 1#)

' To let SOLIDWORKS Design determine an appropriate sheet name, do not set the SheetName property
' importData.SheetName("Model") = "S1"
' importData.SheetName("Sheet2") = "S2"

' Load the specified DXF/DWG file
    Dim newDoc As SldWorks.ModelDoc2
    Set newDoc = swApp.LoadFile4(filename, "", importData, longerrors)

End Sub
```

Import IGES File Example (VBA)

This example shows how to import an IGES file.

```
'-----  
' Preconditions:  
' 1. Verify that the file specified to import exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Imports the specified file into SOLIDWORKS Design.  
' 2. Examine the graphics area, FeatureManager design tree, and Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swImportData As SldWorks.ImportIgesData  
    Dim Err As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.LoadFile4("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\importexport\gasket.igs", "r", swImportData, Err)  
  
    Debug.Print ("Error (0 = no error): " & Err)  
  
End Sub
```

Import Models as Solids Example (VBA)

This example shows how to set the options to import non-native-SOLIDWORKS models into SOLIDWORKS Design as solids.

NOTE: You must set the values of both `swUserPreferenceIntegerValue_e.swCreateBodyFromSurfaces` and `swUserPreferenceIntegerValue_e.swImportUseBrep` for `swUserPreferenceIntegerValue_e.swImportUseBrep` to have an effect.

```

'-----
' Preconditions:
' 1. Substitute the file_to_import.STEP with the
'   name of the STEP file that you want to import.
' 2. Copy the STEP file to this macro's folder.
'-----
' Postconditions:
' 1. Loads the file into a new document
'   and forms solids, if possible, but does
'   not directly map topologies using BREP data.
' 2. Loads the file into a new document
'   and forms solids, if possible, and
'   directly maps topologies using BREP data.
' 3. Examine both documents. Switch documents
'   by clicking Window and clicking the document to view.
' 4. Examine the FeatureManager design tree and
'   the graphics area in each document.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swImportStepData As SldWorks.ImportStepData
    Dim rtn As Boolean
    Dim retVal As Long
    Dim pathname As String
    Dim errors As Long

    Set swApp = Application.SldWorks

    pathname = swApp.GetCurrentMacroPathName
    pathname = Left(pathname, InStrRev(pathname, "\"))

    ' Get swImportUseBrep value
    retVal = swApp.GetUserPreferenceIntegerValue(swImportUseBrep)

    ' Set swImportUseBrep value to 1 (Do not import the model
    ' by directly mapping topologies using BREP data)
    ' Let SOLIDWORKS Design attempt to knit the surfaces into solids
    rtn = swApp.SetUserPreferenceIntegerValue(swImportUseBrep, 1)

    ' Set swCreateBodyFromSurfacesOption to swGeneralImportbyBrep
    rtn = swApp.SetUserPreferenceIntegerValue(SwConst.swUserPreferenceIntegerValue_e.swCreateBodyFromSurfacesOption, SwConst.swGeneralImportByBrep)

    ' Load the STEP file
    swApp.LoadFile4 pathname + "file_to_import.STEP", "r", swImportStepData, errors

    ' Set swImportUseBrep value to 0 (Import the model by directly mapping
    ' topologies using BREP data)
    ' Attempt to import the model by directly mapping topologies
    ' using boundary representation (BREP) data.
    rtn = swApp.SetUserPreferenceIntegerValue(swImportUseBrep, 0)

    ' Get swImportUseBrep value
    retVal = swApp.GetUserPreferenceIntegerValue(swImportUseBrep)

    ' Load the STEP file
    swApp.LoadFile4 pathname + "file_to_import.STEP", "r", swImportStepData, errors

End Sub

```

Import STEP File Example (C#)

This example shows how to import and repair a STEP file.

```
//-----
// Preconditions: Verify that the specified SOLIDWORKS Design part document to open exists.
//
// Postconditions:
// 1. Opens the specified SOLIDWORKS Design part document.
// 2. Saves the part document as a STEP file.
// 3. Closes the part document.
// 4. Imports the STEP file created in step 2.
// 5. Runs import diagnostics on the STEP file and repairs any bad faces.
// 6. Creates Imported1 in the FeatureManager design tree.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace ImportStepDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            PartDoc swPart = default(PartDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            ImportStepData swImportStepData = default(ImportStepData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            string stepFileName = null;

            //Open the part document to export to a STEP file
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\db9 male.sldprt";
            swPart = (PartDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors,
            ref warnings);
            swModel = (ModelDoc2)swPart;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Export the part document to a STEP file
            stepFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\db9 male.STEP";
            status = swModelDocExt.SaveAs(stepFileName, 0, (int)swSaveAsOptions_e.swSaveAsOptions_Silent, null, ref errors, ref warnings);
            swPart = null;

            swApp.CloseDoc("db9 male.sldprt");

            //Get import information
            swImportStepData = (ImportStepData)swApp.GetImportFileData(stepFileName);

            //If ImportStepData::MapConfigurationData is not set, then default to
            //the environment setting swImportStepConfigData; otherwise, override
            //swImportStepConfigData with ImportStepData::MapConfigurationData
            swImportStepData.MapConfigurationData = true;

            //Import the STEP file
            swPart = (PartDoc)swApp.LoadFile4(stepFileName, "r", swImportStepData, ref errors);
            swModel = (ModelDoc2)swPart;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Run diagnostics on the STEP file and repair any bad faces
            errors = swPart.ImportDiagnosis(true, false, true, 0);
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Import STEP File Example (VBA)

This example shows how to import and repair a STEP file.

```
'-----
' Preconditions: Verify that the specified SOLIDWORKS Design part document to open exists.
'
' Postconditions:
' 1. Opens the specified SOLIDWORKS Design part document.
' 2. Saves the part document as a STEP file.
' 3. Closes the part document.
' 4. Imports the STEP file created in step 2.
' 5. Runs import diagnostics on the STEP file and repairs any bad faces.
' 6. Creates Imported1 in the FeatureManager design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swImportStepData As SldWorks.ImportStepData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim stepFileName As String

Sub main()

    Set swApp = Application.SldWorks

    'Open the part document to export to a STEP file
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\db9 male.sldprt"
    Set swPart = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swPart
    Set swModelDocExt = swModel.Extension

    'Export the part document to a STEP file
    stepFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\db9 male.STEP"
    status = swModelDocExt.SaveAs(stepFileName, 0, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)
    Set swPart = Nothing

    swApp.CloseDoc "db9 male.sldprt"

    'Get import information
    Set swImportStepData = swApp.GetImportFileData(stepFileName)

    'If ImportStepData::MapConfigurationData is not set, then default to
    'the environment setting swImportStepConfigData; otherwise, override
    'swImportStepConfigData with ImportStepData::MapConfigurationData
    swImportStepData.MapConfigurationData = True

    'Import the STEP file
    Set swPart = swApp.LoadFile4(stepFileName, "r", swImportStepData, errors)
    Set swModel = swPart
    Set swModelDocExt = swModel.Extension

    'Run diagnostics on the STEP file and repair any bad faces
    errors = swPart.ImportDiagnosis(True, False, True, 0)

End Sub
```

Import STEP File Example (VB.NET)

This example shows how to import and repair a STEP file.

Preconditions: Verify that the specified SOLIDWORKS Design part document to open exists.

Postconditions:

1. Opens the specified SOLIDWORKS Design part document.
2. Saves the part document as a STEP file.
3. Closes the part document.
4. Imports the STEP file created in step 2.
5. Runs import diagnostics on the STEP file and repairs any bad faces.
6. Creates **Imported1** in the FeatureManager design tree.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()

        Dim swPart As PartDoc
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swImportStepData As ImportStepData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim stepFileName As String

        'Open the part document to export to a STEP file
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\db9 male.sldprt"
        swPart = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel = swPart
        swModelDocExt = swModel.Extension

        'Export the part document to a STEP file
        stepFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\db9 male.STEP"
        status = swModelDocExt.SaveAs(stepFileName, 0, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)
        swPart = Nothing

        swApp.CloseDoc("db9 male.sldprt")

        'Get import information
        swImportStepData = swApp.GetImportFileData(stepFileName)

        'If ImportStepData::MapConfigurationData is not set, then default to
        'the environment setting swImportStepConfigData; otherwise, override
        'swImportStepConfigData with ImportStepData::MapConfigurationData
        swImportStepData.MapConfigurationData = True

        'Import the STEP file
        swPart = swApp.LoadFile4(stepFileName, "r", swImportStepData, errors)
        swModel = swPart
        swModelDocExt = swModel.Extension

        'Run diagnostics on the STEP file and repair any bad faces
        errors = swPart.ImportDiagnosis(True, False, True, 0)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
```

```
End Class
```

Include Note in Render File Example (C#)

This example shows how to include a note visible in a model in the render file.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Add a note to the model.
// 3. Verify that.
//    * a ray-trace rendering engine is loaded.
//    * c:\temp exists.
// 4. Open the Immediate window.
//
// Preconditions:
// 1. Sets the option to include the note visible in the model
//    in the render file.
// 2. If prompted to add camera or perspective, click Continue
//    without Camera or Perspective.
// 3. Examine the Immediate window and c:\temp\annotations.png.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace RayTraceRendererCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            RayTraceRenderer swRayTraceRenderer;
            RayTraceRendererOptions swRayTraceRenderOptions;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Access the ray-trace rendering engine
            //swRayTraceRenderer = (RayTraceRenderer)swApp.GetRayTraceRenderer();

            //Set option to include note visible in the model in the render file
            swRayTraceRenderOptions = (RayTraceRendererOptions)swRayTraceRenderer.RayTraceRendererOptions;
            swRayTraceRenderOptions.IncludeAnnotationsInRendering = true;
            Debug.Print("Include note in render file? " + swRayTraceRenderOptions.IncludeAnnotationsInRendering);
            status = swRayTraceRenderer.RenderToFile("c:\\temp\\annotations.png", 0, 0);
            Debug.Print("Note included in render file? " + status);
            status = swRayTraceRenderer.CloseRayTraceRender();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Include Note in Render File Example (VBA)

This example shows how to include a note visible in a model in the render file.

```
'-----
' Preconditions:
' 1. Open a model document.
' 2. Add a note to the model.
' 3. Verify that:
'    * a ray-trace rendering engine is loaded.
'    * c:\temp exists.
' 4. Open the Immediate window.
'
' Preconditions:
' 1. Sets the option to include the note visible in the model
'    in the render file.
' 2. If prompted to add camera or perspective, click Continue
'    without Camera or Perspective.
' 3. Examine the Immediate window and c:\temp\annotations.png.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swRayTraceRenderer As SldWorks.RayTraceRenderer
    Dim swRayTraceRenderOptions As SldWorks.RayTraceRendererOptions
    Dim status As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Access the ray-trace rendering engine
    'Set swRayTraceRenderer = swApp.GetRayTraceRenderer()

    'Set option to include note visible in the model in the render file
    Set swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRendererOptions
    swRayTraceRenderOptions.IncludeAnnotationsInRendering = True
    Debug.Print ("Include note in render file? " & swRayTraceRenderOptions.IncludeAnnotationsInRendering)
    status = swRayTraceRenderer.RenderToFile("c:\temp\annotations.png", 0, 0)
    Debug.Print "Note included in render file? " & status
    status = swRayTraceRenderer.CloseFinalRenderWindow
```

End Sub

Include Note in Render File Example (VB.NET)

This example shows how to include a note visible in a model in the render file.

```

-----
Preconditions:
1. Open a model document.
2. Add a note to the model.
3. Verify that:
   * a ray-trace rendering engine is loaded.
   * c:\temp exists.
4. Open the Immediate window.
Preconditions:
1. Sets the option to include the note visible in the model
   in the render file.
2. If prompted to add camera or perspective, click Continue
   without Camera or Perspective.
3. Examine the Immediate window and c:\temp\annotations.png.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swRayTraceRenderer As RayTraceRenderer
        Dim swRayTraceRenderOptions As RayTraceRenderOptions
        Dim status As Boolean

        swModel = swApp.ActiveDoc

        ' Access the ray-trace rendering engine
        swRayTraceRenderer = swApp.GetRayTraceRenderer()

        'Set option to include note visible in the model in the render file
        swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions
        swRayTraceRenderOptions.IncludeAnnotationsInRendering = True
        Debug.Print("Include note in render file? " & swRayTraceRenderOptions.IncludeAnnotationsInRendering)
        status = swRayTraceRenderer.RenderToFile("c:\temp\annotations.png", 0, 0)
        Debug.Print("Note included in render file? " & status)
        status = swRayTraceRenderer.CloseFinalRenderWindow

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Increase Width of Text Example (VBA)

This example shows how to increase the width of the text in a note by a factor of 2.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.  
' 2. Select the note Fixed Face.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Increases the width of the text in the selected note  
'    by a factor of 2.  
' 2. Examine the graphics area and Immediate window.  
'  
' NOTE: Because the drawing is used elsewhere, do not  
' save changes.  
'-----
```

Sub main()

```
Dim swApp           As SldWorks.SldWorks  
Dim swModel         As SldWorks.modelDoc  
Dim swSelMgr        As SldWorks.SelectionMgr  
Dim swAnnObj        As Object  
Dim swAnnotation    As SldWorks.Annotation  
Dim swTextFormat    As SldWorks.textFormat  
Dim dWidth          As Double  
Dim bRet            As Boolean
```

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swAnnObj = swSelMgr.GetSelectedObject5(1)  
Set swAnnotation = swAnnObj.GetAnnotation  
Set swTextFormat = swAnnotation.GetTextFormat(0)
```

```
dWidth = swTextFormat.WidthFactor  
Debug.Print "Old width = " & dWidth
```

```
swTextFormat.WidthFactor = 2# * dWidth
```

```
bRet = swAnnotation.SetTextFormat(0, False, swTextFormat)  
dWidth = swTextFormat.WidthFactor  
Debug.Print "New width = " & dWidth
```

End Sub

Insert a 3D-Spline Curve Example (VBA)

This example shows how to insert an open 3D-spline curve.

```
'-----  
' Preconditions:  
' 1. Open a new part document.  
' 2. Open a sketch and insert sketch points for  
'    the spline.  
' 3. Close the sketch.  
' 4. Select the sketch points.  
' 5. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a 3D-spline curve.  
' 2. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim nSelCount As Long  
    Dim i As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    nSelCount = swSelMgr.GetSelectedObjectCount  
    Debug.Print "SelCount (before) = " + Str(nSelCount)  
    For i = 1 To nSelCount  
        Debug.Print "    SelType(" + Str(i) + ") = " + Str(swSelMgr.GetSelectedObjectType(i))  
    Next i  
  
    swModel.Insert3DSplineCurve False  
  
    nSelCount = swSelMgr.GetSelectedObjectCount  
    Debug.Print "SelCount (after ) = " + Str(nSelCount)  
    For i = 1 To nSelCount  
        Debug.Print "    SelType(" + Str(i) + ") = " + Str(swSelMgr.GetSelectedObjectType(i))  
    Next i  
  
End Sub
```

Insert a Composite Curve Example (C#)

This example shows how to insert a composite curve using two sketches of splines.

```
//-----
// Preconditions:
// 1. Verify that the part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a sketch of a spline.
// 3. Creates another sketch of a spline.
// 4. Inserts a composite curve using the sketches created
//    in steps 2 and 3.
// 5. Prints the number of joined entities in the composite
//    curve to the Immediate window.
// 6. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CompositeCurveCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            CompositeCurveFeatureData swCompositeCurveFeatureData = default(CompositeCurveFeatureData);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            object pointArray = null;
            double[] points = null;
            bool status = false;
            int nbrEntities = 0;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);

            //Create sketch containing a spline
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            Array.Resize(ref points, 12);
            points[0] = -0.0509020857536935;
            points[1] = 0.0116459784886342;
            points[2] = 0;
            points[3] = -0.0404337904830111;
            points[4] = 0.0249930549587544;
            points[5] = 0;
            points[6] = 0.0396486683377099;
            points[7] = -0.0166184187422084;
            points[8] = 0;
            points[9] = 0.0166184187422084;
            points[10] = -0.0399103757194769;
            points[11] = 0;
            pointArray = points;
            swSketchSegment = swSketchManager.CreateSpline2((pointArray), true);
            swModel.ClearSelection2(true);

            //Create another sketch containing a spline
            swSketchManager.InsertSketch(true);
            Array.Resize(ref points, 12);
            points[0] = -0.0509020857536935;
            points[1] = 0.0116459784886342;
            points[2] = 0;
            points[3] = -0.0370315945200393;
            points[4] = -0.00850548990742951;
            points[5] = 0;
            points[6] = 0.00562670870799184;
            points[7] = -0.014786467069839;
            points[8] = 0;
            points[9] = 0.0166184187422084;
            points[10] = -0.0399103757194769;
            points[11] = 0;
            pointArray = points;
            swSketchSegment = (SketchSegment)swSketchManager.CreateSpline2((pointArray), true);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Insert a composite curve using both sketches
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 1, null, 0);
            swModel.InsertCompositeCurve();

            //Get the number of joined entities in the composite curve
            status = swModelDocExt.SelectByID2("CompCurve1", "REFERENCECURVES", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swCompositeCurveFeatureData = (CompositeCurveFeatureData)swFeature.GetDefinition();
            nbrEntities = swCompositeCurveFeatureData.GetEntitiesToJoinCount();
            Debug.Print("Number of joined entities: " + nbrEntities.ToString());
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Insert a Composite Curve Example (VBA)

This example shows how to insert a composite curve using two sketches of splines.

```
'-----
' Preconditions:
' 1. Verify that the part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a sketch of a spline.
' 3. Creates another sketch of a spline.
' 4. Inserts a composite curve using the sketches created
'    in steps 2 and 3.
' 5. Prints the number of joined entities in the composite
'    curve to the Immediate window.
' 6. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketchManager As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swCompositeCurveFeatureData As SldWorks.CompositeCurveFeatureData
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim pointArray As Variant
Dim points() As Double
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prt", 0, 0, 0)

    'Create sketch containing a spline
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    ReDim points(0 To 11) As Double
    points(0) = -5.09020857536935E-02
    points(1) = 1.16459784886342E-02
    points(2) = 0
    points(3) = -4.04337904830111E-02
    points(4) = 2.49930549587544E-02
    points(5) = 0
    points(6) = 3.96486683377099E-02
    points(7) = -1.66184187422084E-02
    points(8) = 0
    points(9) = 1.66184187422084E-02
    points(10) = -3.99103757194769E-02
    points(11) = 0
    pointArray = points
    Set swSketchSegment = swSketchManager.CreateSpline2((pointArray), True)
    swModel.ClearSelection2 True

    'Create another sketch containing a spline
    swSketchManager.InsertSketch True
    ReDim points(0 To 11) As Double
    points(0) = -5.09020857536935E-02
    points(1) = 1.16459784886342E-02
    points(2) = 0
    points(3) = -3.70315945200393E-02
    points(4) = -8.50548990742951E-03
    points(5) = 0
    points(6) = 5.62670870799184E-03
    points(7) = -0.014786467069839
    points(8) = 0
    points(9) = 1.66184187422084E-02
    points(10) = -3.99103757194769E-02
    points(11) = 0
    pointArray = points
    Set swSketchSegment = swSketchManager.CreateSpline2((pointArray), True)
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    'Insert a composite curve using both sketches
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)
    swModel.InsertCompositeCurve

    'Get the number of joined entities in the composite curve
    status = swModelDocExt.SelectByID2("CompCurve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swCompositeCurveFeatureData = swFeature.GetDefinition
    Debug.Print ("Number of joined entities: " & swCompositeCurveFeatureData.GetEntitiesToJoinCount)

End Sub
```

Insert a Composite Curve Example (VB.NET)

This example shows how to insert a composite curve using two sketches of splines.

```

-----
Preconditions:
1. Verify that the part document template exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part document.
2. Creates a sketch of a spline.
3. Creates another sketch of a spline.
4. Inserts a composite curve using the sketches created
   in steps 2 and 3.
5. Prints the number of joined entities in the composite
   curve to the Immediate window.
6. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSketchSegment As SketchSegment
        Dim swSketchManager As SketchManager
        Dim swModelDocExt As ModelDocExtension
        Dim swCompositeCurveFeatureData As CompositeCurveFeatureData
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim pointArray As Object
        Dim points() As Double
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

        'Create sketch containing a spline
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        ReDim Preserve points(11)
        points(0) = -0.0509020857536935
        points(1) = 0.0116459784886342
        points(2) = 0
        points(3) = -0.0404337904830111
        points(4) = -0.0249930549587544
        points(5) = 0
        points(6) = 0.0396486683377099
        points(7) = -0.0166184187422084
        points(8) = 0
        points(9) = 0.0166184187422084
        points(10) = -0.0399103757194769
        points(11) = 0
        pointArray = points
        swSketchSegment = swSketchManager.CreateSpline2((pointArray), True)
        swModel.ClearSelection2(True)

        'Create another sketch containing a spline
        swSketchManager.InsertSketch(True)
        ReDim Preserve points(11)
        points(0) = -0.0509020857536935
        points(1) = 0.0116459784886342
        points(2) = 0
        points(3) = -0.0370315945200393
        points(4) = -0.00850548990742951
        points(5) = 0
        points(6) = 0.00562670870799184
        points(7) = -0.014786467069839
        points(8) = 0
        points(9) = 0.0166184187422084
        points(10) = -0.0399103757194769
        points(11) = 0
        pointArray = points
        swSketchSegment = swSketchManager.CreateSpline2((pointArray), True)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        'Insert a composite curve using both sketches
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 1, Nothing, 0)
        swModel.InsertCompositeCurve()

        'Get the number of joined entities in the composite curve
        status = swModelDocExt.SelectByID2("CompCurve1", "REFERENCECURVES", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swCompositeCurveFeatureData = swFeature.GetDefinition
        Debug.Print("Number of joined entities: " & swCompositeCurveFeatureData.GetEntitiesToJoinCount)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert a Note Example (C#)

This example shows how to insert a geometric tolerance symbol in an active drawing document.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
//
// Postconditions:
// 1. Inserts a geometric tolerance symbol is inserted at the specified position.
// 2. Examine the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SetPosition2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            Annotation Annotation = default(Annotation);
            object swSelobj2 = null;
            SelectionMgr swSelMgr = default(SelectionMgr);
            Note Note = default(Note);
            bool boolstatus = false;
            int longstatus = 0;

            Part = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)Part.SelectionManager;
            boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.166288048468037, 0.223859686746988,
            -0.000400000000013279, false, 0, null, 0);
            swSelobj2 = swSelMgr.GetSelectedObject6(1, -1);
            Note = (Note)Part.InsertNote("<MOD-CL>");
            if ((Note != null))
            {
                Note.Angle = 0;
                boolstatus = Note.SetBalloon(0, 0);
                Annotation = (Annotation)Note.GetAnnotation();
                object[] AttEntArr = new object[1];
                AttEntArr[0] = swSelobj2;
                object vAttEntArrIn = null;
                vAttEntArrIn = AttEntArr;
                boolstatus = Annotation.SetAttachedEntities(vAttEntArrIn);
                if ((Annotation != null))
                {
                    longstatus = Annotation.SetLeader3(1, 0, true, true, false, false);
                    boolstatus = Annotation.SetPosition2(0.1038962799325, 0.135343450253, 0);
                }
            }
            Part.ClearSelection2(true);
            Part.WindowRedraw();
        }

        public SldWorks swApp;
    }
}
```

Insert a Note Example (VBA)

This example shows how to insert a geometric tolerance symbol in an active drawing document.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Inserts a geometric tolerance symbol at the specified position.
' 2. Examine the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2
    Dim Annotation As SldWorks.Annotation
    Dim swSelobj2 As Object
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim Note As SldWorks.Note
    Dim boolstatus As Boolean
    Dim longstatus As Long

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    Set swSelMgr = Part.SelectionManager
    boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.166288048468037, 0.223859686746988, -4.00000000013279E-04, False, 0, Nothing, 0)
    Set swSelobj2 = swSelMgr.GetSelectedObject6(1, -1)
    Set Note = Part.InsertNote("<MOD-CL>")
    If Not Note Is Nothing Then
        Note.Angle = 0
        boolstatus = Note.SetBalloon(0, 0)

        Set Annotation = Note.GetAnnotation()

        Dim AttEntArr(0) As Object
        Set AttEntArr(0) = swSelobj2
        Dim vAttEntArrIn As Variant
        vAttEntArrIn = AttEntArr
        boolstatus = Annotation.SetAttachedEntities(vAttEntArrIn)
        If Not Annotation Is Nothing Then
            longstatus = Annotation.SetLeader3(1, 0, True, True, False, False)
            boolstatus = Annotation.SetPosition2(0.1038962799325, 0.135343450253, 0)
        End If
    End If

    End If

    Part.ClearSelection2 True
    Part.WindowRedraw
End Sub

```

Insert a Note Example (VB.NET)

This example shows how to insert a geometric tolerance symbol in an active drawing document.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Inserts a a geometric tolerance symbol at the specified position.
' 2. Examine the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Partial Class SolidWorksMacro
    Sub main()
        Dim Part As ModelDoc2
        Dim Annotation As Annotation
        Dim swSelobj2 As Object
        Dim swSelMgr As SelectionMgr
        Dim Note As Note
        Dim boolstatus As Boolean
        Dim longstatus As Integer

        Part = swApp.ActiveDoc
        swSelMgr = Part.SelectionManager
        boolstatus = Part.Extension.SelectByID2("", "EDGE", 0.166288048468037, 0.223859686746988, -0.000400000000013279,
        False, 0, Nothing, 0)
        swSelobj2 = swSelMgr.GetSelectedObject6(1, -1)
        Note = Part.InsertNote("<MOD-CL>")
        If Not Note Is Nothing Then
            Note.Angle = 0
            boolstatus = Note.SetBalloon(0, 0)
            Annotation = Note.GetAnnotation()

            Dim AttEntArr(0) As Object
            AttEntArr(0) = swSelobj2
            Dim vAttEntArrIn As Object
            vAttEntArrIn = AttEntArr
            boolstatus = Annotation.SetAttachedEntities(vAttEntArrIn)
            If Not Annotation Is Nothing Then
                longstatus = Annotation.SetLeader3(1, 0,
                True, True, False, False)
            boolstatus = Annotation.SetPosition2(0.1038962799325, 0.135343450253, 0)
        End If
    End Sub
End Class

```

Insert Variable Pattern Feature Example (C#)

This example shows how to insert a variable pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part to open exists.
// 2. Verify that c:\temp exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a variable pattern feature.
// 2. Exports the table to a Microsoft Excel file.
// 3. Examine the graphics area, Immediate window, and c:\temp.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            DimPatternFeatureData swDimensionDrivenTablePatternFeat = default(DimPatternFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int nbr = 0;
            int dimNbr = 0;
            int i = 0;
            int j = 0;
            string[] controllingDimNames = null;
            string controllingDimName = null;
            string instanceName = null;
            string[] instanceNames = null;
            string patternName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\cstick.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select feature to pattern
            status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);

            //Select reference geometry to drive seed feature
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 1048576, null, 0);

            //Populate table
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPRT", 256, 1, 0, 1, 0, 0.085);
            status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPRT", 256, 1, 0, 1, 0, 0.04);
            status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPRT", 256, 1, 0, 1, 0, 0.03);
            status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPRT", 256, 1, 0, 2, 0, 0.105);
            status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPRT", 256, 1, 0, 2, 0, 0.06);
            status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPRT", 256, 1, 0, 2, 0, 0.05);

            //Insert the variable pattern feature
            //and access its feature data
            swDimensionDrivenTablePatternFeat = (DimPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmDimPattern);
            swFeature = (Feature)swFeatureManager.CreateFeature(swDimensionDrivenTablePatternFeat);
            swDimensionDrivenTablePatternFeat = (DimPatternFeatureData)swFeature.GetDefinition();
            status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, null);

            nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount();
            Debug.Print("Number of pattern instances: " + nbr);

            dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount();
            Debug.Print("Number of controlling dimensions: " + dimNbr);
            Debug.Print("Controlling dimension names: ");
            controllingDimNames = new string[dimNbr + 1];
            for (i = 0; i < dimNbr; i++)
            {
                controllingDimNames[i] = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i);
                controllingDimName = controllingDimNames[i];
                Debug.Print(" " + controllingDimName);
            }

            Debug.Print("Names of pattern instances:");
            instanceNames = new string[nbr];
            j = 0;
            for (i = 1; i <= nbr; i++)
            {
                instanceName = swDimensionDrivenTablePatternFeat.GetInstanceNameByIndex(i);
                Debug.Print(" " + instanceName);
                instanceNames[j] = instanceName;
                j = j + 1;
            }

            Debug.Print("Table row indices of pattern instances:");
            for (i = 0; i <= nbr - 1; i++)
            {
                Debug.Print(" " + swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames[i]) + ": " + instanceNames[i]);
            }
        }
    }
}
```

```

        Debug.Print("Pattern dimension names: ");
        for (i = 0; i < nbr; i++)
        {
            for (j = 1; j < dimNbr; i++)
            {
                patternName = swDimensionDrivenTablePatternFeat.GetInstanceDimensionName(instanceNames[i], controllingDimNames[j]);
            }
            Debug.Print(" " + patternName + ": " + instanceNames[i] + ": Controlling dimension name: " + swDimensionDrivenTablePatternFeat.GetControllingDimensionName(j));
        }

        Debug.Print("Pattern instance suppression state:");
        for (i = 0; i <= nbr - 1; i++)
        {
            Debug.Print(" " + swDimensionDrivenTablePatternFeat.GetInstanceSuppressStateByIndex(swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames[i])) + ": " + instanceNames[i]);
        }

        //Export table to Microsoft Excel file
        errors = swDimensionDrivenTablePatternFeat.ExportToExcel("c:\\temp\\cstickVarPattern.xls", true);
        Debug.Print ("Table exported to Microsoft Excel file (0 = Succeeded)? " + errors);

        swDimensionDrivenTablePatternFeat.ReleaseSelectionAccess();
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Insert Variable Pattern Feature Example (VBA)

This example shows how to insert a variable pattern feature.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Verify that c:\temp exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a variable pattern feature.
' 2. Exports the table to a Microsoft Excel file.
' 3. Examine the graphics area, Immediate window, and c:\temp.
'
' NOTE: Because the part is used elsewhere, do not changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swDimensionDrivenTablePatternFeat As SldWorks.DimPatternFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long, warnings As Long
Dim nbr As Long
Dim dimNbr As Long
Dim i As Long
Dim j As Long
Dim controllingDimNames() As String
Dim controllingDimName As String
Dim instanceName As String
Dim instanceNames() As String
Dim patternName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cstick.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Select feature to pattern
    status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)

    'Select reference geometry to drive seed feature
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 1048576, Nothing, 0)

    'Populate table
    Set swFeatureManager = swModel.FeatureManager
    status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.085)
    status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.04)
    status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.03)
    status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.105)
    status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.06)
    status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.05)

    'Insert the variable pattern feature and access its feature data

    Set swDimensionDrivenTablePatternFeat = swFeatureManager.CreateDefinition(swFmdDimPattern)
    Set swFeature = swFeatureManager.CreateFeature(swDimensionDrivenTablePatternFeat)

    Set swDimensionDrivenTablePatternFeat = swFeature.GetDefinition
    status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

    nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
    Debug.Print ("Number of pattern instances: " & nbr)

    dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
    Debug.Print ("Number of controlling dimensions: " & dimNbr)
    Debug.Print (" Controlling dimension names: ")
    ReDim Preserve controllingDimNames(dimNbr)
    For i = 0 To dimNbr - 1
        controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
        controllingDimName = controllingDimNames(i)
        Debug.Print (" " & controllingDimName)
    Next i

    Debug.Print ("Names of pattern instances:")
    ReDim Preserve instanceNames(nbr - 1)
    j = 0
    For i = 1 To nbr
        instanceName = swDimensionDrivenTablePatternFeat.GetInstanceNameByIndex(i)
        Debug.Print (" " & instanceName)
        instanceNames(j) = instanceName
        j = j + 1
    Next i

    Debug.Print ("Table row indices of pattern instances:")
    For i = 0 To nbr - 1
        Debug.Print (" " & swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames(i)) & ": " & instanceNames(i))
    Next i
```

```

Debug.Print ("Pattern dimension names: ")
For i = 0 To nbr - 1
    For j = 1 to dimNbr -1
        patternName = swDimensionDrivenTablePatternFeat.GetInstanceDimensionName(instanceNames(i), controllingDimNames(j))
        Debug.Print (" " & patternName & ": " & instanceNames(i) & ": Controlling dimension name: " &
swDimensionDrivenTablePatternFeat.GetControllingDimensionName(j))
    Next j
Next i

Debug.Print ("Pattern instance suppression state:")
For i = 0 To nbr - 1
    Debug.Print (" " &
swDimensionDrivenTablePatternFeat.GetInstanceSupressStateByIndex(swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames(i))) & ": " &
instanceNames(i))
Next i

' Export table to Microsoft Excel file
errors = swDimensionDrivenTablePatternFeat.ExportToExcel("c:\temp\cstickVarPattern.xls", True)
Debug.Print "Table exported to Microsoft Excel file (0 = Succeeded)? " & errors

swDimensionDrivenTablePatternFeat.ReleaseSelectionAccess

End Sub

```

Insert Variable Pattern Feature Example (VB.NET)

This example shows how to insert a variable pattern feature.

```

Preconditions:
1. Verify that the specified part to open exists.
2. Verify that c:\temp exists.
3. Open the Immediate window.

Postconditions:
1. Inserts a variable pattern feature.
2. Exports the table to a Microsoft Excel file.
3. Examine the graphics area, Immediate window, and c:\temp.

NOTE: Because the part is used elsewhere, do not save changes.

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
Public Sub Main()
```

```

Dim swModel As ModelDoc2
Dim swModelDocExt As ModelDocExtension
Dim swFeatureManager As FeatureManager
Dim swFeature As Feature
Dim swDimensionDrivenTablePatternFeat As DimPatternFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Integer, warnings As Integer
Dim nbr As Integer
Dim dimNbr As Integer
Dim i As Integer

```

```

Dim j As Integer
Dim controllingDimNames() As String
Dim controllingDimName As String
Dim instanceName As String
Dim instanceNames() As String
Dim patternName As String

```

```

fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\cstick.sldprt"
swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warr
swModelDocExt = swModel.Extension

```

```

'Select feature to pattern
status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)

```

```

'Select reference geometry to drive seed feature
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 1048576, Nothing, 0)

```

```

'Populate table
swFeatureManager = swModel.FeatureManager
status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.085)
status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.04)
status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPR", 256, 1, 0, 1, 0, 0.03)
status = swFeatureManager.InsertVaryInstanceOverride("D1@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.105)
status = swFeatureManager.InsertVaryInstanceOverride("D3@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.06)
status = swFeatureManager.InsertVaryInstanceOverride("D4@Sketch2@cstick.SLDPR", 256, 1, 0, 2, 0, 0.05)

```

```

'Insert the variable pattern feature
'and access its feature data
swDimensionDrivenTablePatternFeat = swFeatureManager.CreateDefinition(swFeatureNameID_e.swFmDimPattern)
swFeature = swFeatureManager.CreateFeature(swDimensionDrivenTablePatternFeat)
swDimensionDrivenTablePatternFeat = swFeature.GetDefinition
status = swDimensionDrivenTablePatternFeat.AccessSelections(swModel, Nothing)

```

```

nbr = swDimensionDrivenTablePatternFeat.GetInstanceCount
Debug.Print("Number of pattern instances: " & nbr)

```

```

dimNbr = swDimensionDrivenTablePatternFeat.GetControllingDimensionCount
Debug.Print("Number of controlling dimensions: " & dimNbr)
Debug.Print("Controlling dimension names: ")
ReDim controllingDimNames(dimNbr)
For i = 0 To dimNbr - 1
controllingDimNames(i) = swDimensionDrivenTablePatternFeat.GetControllingDimensionName(i)
controllingDimName = controllingDimNames(i)
Debug.Print(" " & controllingDimName)
Next i

```

```

Debug.Print("Names of pattern instances:")
ReDim instanceNames(nbr - 1)
j = 0
For i = 1 To nbr
instanceName = swDimensionDrivenTablePatternFeat.GetInstanceNameByIndex(i)
Debug.Print(" " & instanceName)

```

```

        instanceNames(j) = instanceName
        j = j + 1
    Next i

    Debug.Print("Table row indices of pattern instances:")
    For i = 0 To nbr - 1
        Debug.Print(" " & swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames(i)) & ": " & instanceNames(i))
    Next i

    Debug.Print("Pattern dimension names: ")
    For i = 0 To nbr - 1
        For j = 0 To dimNbr - 1
            patternName = swDimensionDrivenTablePatternFeat.GetInstanceDimensionName(instanceNames(i), controllingDimNames(j))
        Next j
    Next i

    Debug.Print(" " & patternName & ": " & instanceNames(i) & ": Controlling dimension name: " & swDimensionDrivenTablePatternFeat.GetControllingDimensionName(j))
    Next j
Next i

    Debug.Print("Pattern instance suppression state:")
    For i = 0 To nbr - 1
        Debug.Print(" " & swDimensionDrivenTablePatternFeat.GetInstanceSuppressStateByIndex(swDimensionDrivenTablePatternFeat.GetTableRowIndex(instanceNames(i))) & ": " & instanceNames(i))
    Next i

    ' Export table to Microsoft Excel file
    errors = swDimensionDrivenTablePatternFeat.ExportToExcel("c:\temp\cstickVarPattern.xls", True)
    Debug.Print "Table exported to Microsoft Excel file (0 = Succeeded)? " & errors

    swDimensionDrivenTablePatternFeat.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Insert Alternate Position View Example (VBA)

This example shows how to insert an **Alternate Position View**.

```
'-----  
'  
' Preconditions: Drawing sheet is open with a  
'           drawing view selected.  
'  
' Postconditions: An alternate position view is created.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swView As SldWorks.View  
Dim swSelMgr As SldWorks.SelectionMgr  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Select the drawing on which to superimpose  
    ' an Alternate Position View  
    Set swView = swSelMgr.GetSelectedObject6(1, 0)  
  
    ' Insert the Alternate Position View and  
    ' create the configuration called Configxxx  
    Set swView = swView.InsertAlternateView("Configxxx")  
  
    ' Print the type of view; should be 10, which  
    ' is an Alternate Position View  
    If Not swView Is Nothing Then  
        Debug.Print "Type of view: " & swView.Type  
    End If  
  
End Sub
```

Insert and Access Fold Feature Example (C#)

This example shows how to insert and access a fold feature.

```
//-----
// Postconditions:
// 1. Verify that the specified sheet metal part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified sheet metal part document.
// 2. Creates an unfold feature.
// 3. Creates a fold feature.
// 4. Prints to the Immediate window some fold feature data.
// 5. Examine the FeatureManager design tree and the Immediate window.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FoldsFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            FoldsFeatureData swFoldsFeatureData = default(FoldsFeatureData);
            Face2 swFace = default(Face2);
            Body2 swBody = default(Body2);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int i = 0;
            object[] bendsArray = null;

            //Open sheet metal part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Insert unfold feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID("", "FACE", 0.0135437392197275, 0.013831948116092, 0.0180159642212061, true, 0, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0.0139765211971223, 0.045779599797811, -0.018375967305019, true, 0, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 0.0145403568253926, 0.0461305825900808, -0.00849880301666417, true, 0, null, 0);

            status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0.013808065447904, 0.0455785871991452, 0.0109703538056465, true, 0, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 0.0139037479688966, 0.0457015473971296, 0.0275647689667267, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("", "FACE", 0.0135437392197275, 0.013831948116092, 0.0180159642212061, false, 1, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0.0139765211971223, 0.045779599797811, -0.018375967305019, true, 4, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 0.0145403568253926, 0.0461305825900808, -0.00849880301666417, true, 4, null, 0);

            status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0.013808065447904, 0.0455785871991452, 0.0109703538056465, true, 4, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 0.0139037479688966, 0.0457015473971296, 0.0275647689667267, true, 4, null, 0);
            swModel.InsertSheetMetalUnfold();

            //Insert fold feature
            status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0.0135437392197559, 0.0460611937937756, -0.019419982567797, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Unfold1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0.0135437392197559, 0.0460611937937756, -0.019419982567797, true, 4, null, 0);
            swModel.InsertSheetMetalFold();

            //Access the fold feature
            status = swModelDocExt.SelectByID("Fold1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swFoldsFeatureData = (FoldsFeatureData)swFeature.GetDefinition();
            status = swFoldsFeatureData.AccessSelections(swModel, null);

            //Get name of fixed face body in the fold feature
            swFace = (Face2)swFoldsFeatureData.FixedFace;
            swBody = (Body2)swFace.GetBody();
            Debug.Print("Name of the body of the fixed face of the fold feature: " + swBody.Name);

            //Get the names bend features in the fold feature
            bendsArray = (object[])swFoldsFeatureData.Bends;
            for (i = 0; i < bendsArray.Length; i++)
            {
                swFeature = (Feature)bendsArray[i];
                Debug.Print("Name of bend feature" + (i + 1) + " of the fold feature: " + swFeature.Name);
            }

            //Release selection access
            swFoldsFeatureData.ReleaseSelectionAccess();
        }
    }
}

/// <summary>
```

```
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Insert and Access Fold Feature Example (VBA)

This example shows how to insert and access a fold feature.

```

'-----
' Postconditions:
' 1. Verify that the specified sheet metal part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified sheet metal part document.
' 2. Creates an unfold feature.
' 3. Creates a fold feature.
' 4. Prints to the Immediate window some fold feature data.
' 5. Examine the FeatureManager design tree and the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFoldsFeatureData As SldWorks.FoldsFeatureData
Dim swFace As SldWorks.Face2
Dim swBody As SldWorks.Body2
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim i As Long
Dim bendsArray As Variant

Sub main()

    Set swApp = Application.SldWorks

    'Open sheet metal part
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Insert unfold feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", 1.35437392197275E-02, 0.013831948116092, 1.80159642212061E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 1.39765211971223E-02, 0.045779599797811, -0.018375967305019, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 1.45403568253926E-02, 4.61305825900808E-02, -8.49880301666417E-03, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0.013808065447904, 4.55785871991452E-02, 1.09703538056465E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 1.39037479688966E-02, 4.57015473971296E-02, 2.75647689667267E-02, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "FACE", 1.35437392197275E-02, 0.013831948116092, 1.80159642212061E-02, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 1.39765211971223E-02, 0.045779599797811, -0.018375967305019, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 1.45403568253926E-02, 4.61305825900808E-02, -8.49880301666417E-03, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0.013808065447904, 4.55785871991452E-02, 1.09703538056465E-02, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 1.39037479688966E-02, 4.57015473971296E-02, 2.75647689667267E-02, True, 4, Nothing, 0)
    swModel.InsertSheetMetalUnfold

    'Insert fold feature
    status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 1.35437392197559E-02, 4.60611937937756E-02, -0.019419982567797, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Unfold1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 1.35437392197559E-02, 4.60611937937756E-02, -0.019419982567797, True, 4, Nothing, 0)
    swModel.InsertSheetMetalFold

    'Access the fold feature
    status = swModelDocExt.SelectByID("Fold1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swFoldsFeatureData = swFeature.GetDefinition
    status = swFoldsFeatureData.AccessSelections(swModel, Nothing)
    'Get name of fixed face body in the fold feature
    Set swFace = swFoldsFeatureData.FixedFace
    Set swBody = swFace.GetBody
    Debug.Print "Name of the body of the fixed face of the fold feature: " & swBody.Name
    'Get the names bend features in the fold feature
    bendsArray = swFoldsFeatureData.Bends
    For i = 0 To UBound(bendsArray)
        Set swFeature = bendsArray(i)
        Debug.Print "Name of bend feature" & i + 1 & " of the fold feature: " & swFeature.Name
    Next i
    'Release selection access
    swFoldsFeatureData.ReleaseSelectionAccess

End Sub

```

Insert and Access Fold Feature Example (VB.NET)

This example shows how to insert and access a fold feature.

```

'-----
Postconditions:
1. Verify that the specified sheet metal part document exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified sheet metal part document.
2. Creates an unfold feature.
3. Creates a fold feature.
4. Prints to the Immediate window some fold feature data.
5. Examine the FeatureManager design tree and the Immediate window.

NOTE: Because this part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swFoldsFeatureData As FoldsFeatureData
        Dim swFace As Face2
        Dim swBody As Body2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer
        Dim bendsArray() As Object

        'Open sheet metal part
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Insert unfold feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend4", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend5", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend6", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
        swModel.InsertSheetMetalUnfold()

        'Insert fold feature
        status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Unfold1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "FACE", 0, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("EdgeBend3", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
        swModel.InsertSheetMetalFold()

        'Access the fold feature
        status = swModelDocExt.SelectByID("Fold1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swFoldsFeatureData = swFeature.GetDefinition
        status = swFoldsFeatureData.AccessSelections(swModel, Nothing)

        'Get name of fixed face body in the fold feature
        swFace = swFoldsFeatureData.FixedFace
        swBody = swFace.GetBody
        Debug.Print("Name of the body of the fixed face of the fold feature: " & swBody.Name)

        'Get the names bend features in the fold feature
        bendsArray = swFoldsFeatureData.Bends
        For i = 0 To UBound(bendsArray)
            swFeature = bendsArray(i)
            Debug.Print("Name of bend feature" & (i + 1) & " of the fold feature: " & swFeature.Name)
        Next i

        'Release selection access
        swFoldsFeatureData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert and Change DeleteFace Feature Example (C#)

This example shows how to insert a DeleteFace feature and how to modify that feature.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates and modifies a DeleteFace feature.
// 2. Examine the Immediate window.
//
// NOTE: Because this part document is used elsewhere, do not save changes.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace DeleteFaceFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            DeleteFaceFeatureData swDeleteFaceFeature = default(DeleteFaceFeatureData);
            string featureName = null;
            bool boolstatus = false;
            int opt = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select a face for the
            // DeleteFace feature
            boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.002251015125069, -0.001872569429423, 0.02015405789763, false, 0,
            null, 0);

            // Create a DeleteFace feature
            boolstatus = swModelDocExt.InsertDeleteFace((int)swFaceDeleteOption_e.swFaceDelete_Default);

            // Get the DeleteFace feature
            swFeature = (Feature)swModel.FirstFeature();
            while ((swFeature != null)) {
                featureName = swFeature.Name;
                while (featureName != "DeleteFace1") {
                    swFeature = (Feature)swFeature.GetNextFeature();
                    featureName = swFeature.Name;
                }

                Debug.Print("Feature name: " + featureName);
                swDeleteFaceFeature = (DeleteFaceFeatureData)swFeature.GetDefinition();
                boolstatus = swDeleteFaceFeature.AccessSelections(swModel, null);
                Debug.Print(" Number of deleted faces: " + swDeleteFaceFeature.GetDeletedFacesCount());

                // Get the DeleteFace feature's option
                opt = swDeleteFaceFeature.Options;
                Debug.Print(" Before changing the option...");
                DeleteFaceOptions(opt);

                // Change the DeleteFace feature's option
                swDeleteFaceFeature.Options = (int)swFaceDeleteOption_e.swFaceDelete_Patch;
                opt = swDeleteFaceFeature.Options;
                Debug.Print(" After changing the option...");
                DeleteFaceOptions(opt);

                // Save modification made to DeleteFace feature
                boolstatus = swFeature.ModifyDefinition(swDeleteFaceFeature, swModel, null);

                // Get next feature until no more features
                swFeature = (Feature)swFeature.GetNextFeature();
            }
        }

        public void DeleteFaceOptions(long options)
        {

```

```

switch (options) {
    case 0:
        Debug.Print(" Option = swFaceDelete_Default");
        break;
    case 1:
        Debug.Print(" Option = swFaceDelete_Patch");
        break;
    case 2:
        Debug.Print(" Option = swFaceDelete_Fill");
        break;
    case 3:
        Debug.Print(" Option = swFaceDelete_FillWithTangent");
        break;
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Insert and Change DeleteFace Feature Example (VBA)

This example shows how to insert a DeleteFace feature and how to modify that feature.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates and modifies a DeleteFace feature.
' 2. Examine the FeatureManager design tree, the graphics
'    area, and the Immediate window.
'
' NOTE: Because this part document is used elsewhere, do not save changes.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swDeleteFaceFeature As SldWorks.DeleteFaceFeatureData
Dim featureName As String
Dim boolstatus As Boolean
Dim opt As Long

Sub main()

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension

' Select a face for the DeleteFace
' feature
boolstatus = swModelDocExt.SelectByID("", "FACE", 0.002251015125069, -0.001872569429423, 0.02015405789763, False, 0, Nothing, 0)

' Insert a DeleteFace feature
boolstatus = swModelDocExt.InsertDeleteFace(swFaceDelete_Default)

' Get the DeleteFace feature
Set swFeature = swModel.FirstFeature

While Not swFeature Is Nothing
    featureName = swFeature.Name
    While featureName <> "DeleteFace1"
        Set swFeature = swFeature.GetNextFeature
        featureName = swFeature.Name
    Wend
    Debug.Print "Feature name: " & featureName
    Set swDeleteFaceFeature = swFeature.GetDefinition
    boolstatus = swDeleteFaceFeature.AccessSelections(swModel, Nothing)
    Debug.Print "    Number of deleted faces: " & swDeleteFaceFeature.GetDeletedFacesCount

    ' Get the DeleteFace feature's option
    opt = swDeleteFaceFeature.options
    Debug.Print "    Before changing the option..."
    DeleteFaceOptions opt

    ' Change the DeleteFace feature's option
    swDeleteFaceFeature.options = swFaceDelete_Patch
    opt = swDeleteFaceFeature.options
    Debug.Print "    After changing the option..."
    DeleteFaceOptions opt

    ' Save modification made to DeleteFace feature
    boolstatus = swFeature.ModifyDefinition(swDeleteFaceFeature, swModel, Nothing)

    ' Get next feature until no more features
    Set swFeature = swFeature.GetNextFeature
Wend

End Sub

Sub DeleteFaceOptions(options As Long)
    Select Case options
        Case 0
            Debug.Print "    Option = swFaceDelete_Default"
        Case 1
            Debug.Print "    Option = swFaceDelete_Patch"
        Case 2
            Debug.Print "    Option = swFaceDelete_Fill"
        Case 3
            Debug.Print "    Option = swFaceDelete_FillWithTangent"
    End Select
End Sub
```

Insert and Change DeleteFace Feature Example (VB.NET)

This example shows how to insert a DeleteFace feature and how to then modify that feature.

```

' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\fillets\knob.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates and modifies a DeleteFace feature.
' 2. Examine the Immediate window.
'
' NOTE: Because this part document is used elsewhere, do not save changes.
' -----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swDeleteFaceFeature As DeleteFaceFeatureData
        Dim featureName As String
        Dim boolstatus As Boolean
        Dim opt As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Select a face for the
        ' DeleteFace feature
        boolstatus = swModel.Extension.SelectByID("", "FACE", 0.002251015125069, -0.001872569429423, 0.02015405789763, False, 0,
Nothing, 0)

        ' Create a DeleteFace feature
        boolstatus = swModelDocExt.InsertDeleteFace(swFaceDeleteOption_e.swFaceDelete_Default)
        ' Get the DeleteFace feature
        swFeature = swModel.FirstFeature
        While Not swFeature Is Nothing
            featureName = swFeature.Name
            While featureName <> "DeleteFace1"
                swFeature = swFeature.GetNextFeature
                featureName = swFeature.Name
            End While
            Debug.Print("Feature name: " & featureName)
            swDeleteFaceFeature = swFeature.GetDefinition
            boolstatus = swDeleteFaceFeature.AccessSelections(swModel, Nothing)
            Debug.Print(" Number of deleted faces: " & swDeleteFaceFeature.GetDeletedFacesCount)

            ' Get the DeleteFace feature's option
            opt = swDeleteFaceFeature.Options
            Debug.Print(" Before changing the option...")
            DeleteFaceOptions(opt)

            ' Change the DeleteFace feature's option
            swDeleteFaceFeature.Options = swFaceDeleteOption_e.swFaceDelete_Patch
            opt = swDeleteFaceFeature.Options
            Debug.Print(" After changing the option...")
            DeleteFaceOptions(opt)

            ' Save modification made to DeleteFace feature
            boolstatus = swFeature.ModifyDefinition(swDeleteFaceFeature, swModel, Nothing)
            ' Get next feature until no more features

            swFeature = swFeature.GetNextFeature
        End While

    End Sub

    Sub DeleteFaceOptions(ByVal options As Long)
        Select Case options
            Case 0
                Debug.Print(" Option = swFaceDelete_Default")
            Case 1
                Debug.Print(" Option = swFaceDelete_Patch")
            Case 2
                Debug.Print(" Option = swFaceDelete_Fill")
            Case 3
                Debug.Print(" Option = swFaceDelete_FillWithTangent")
        End Select
    End Sub
End Class

```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Insert and Modify Datum Target Symbol Example (C#)

This example shows how to insert and modify a datum target symbol.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Inserts a datum target symbol on the selected
//    face.
// 3. Gets and modifies the datum target symbol's reference label.
// 4. Examine the Immediate window and graphics area.
//
// NOTE: Because the part document is used elsewhere,
// do not save any changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DatumTargetSymCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DatumTargetSym swDatumTargetSym = default(DatumTargetSym);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\holecube.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Select face for datum target symbol
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", -0.00723565448987529, -0.0259480787517532, 0, false, 0, null, 0);

            //Insert datum target symbol
            swDatumTargetSym = (DatumTargetSym)swModelDocExt.InsertDatumTargetSymbol3("a", "", "", 0, false, 0.003, 0.03, "3", "", true,
12, 0, true, false, true, (int)swMoveableDatumStyle_e.swMoveableDatumStyle_Horizontal);

            //Get and set datum reference label
            Debug.Print("Current reference label: " + swDatumTargetSym.GetDatumReferenceLabel(0));
            status = swDatumTargetSym.SetDatumReferenceLabel(0, "b");
            Debug.Print("Modified reference label: " + swDatumTargetSym.GetDatumReferenceLabel(0));
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert and Modify Datum Target Symbol Example (VBA)

This example shows how to insert and modify a datum target symbol.

```
'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Inserts a datum target symbol on the selected
'    face.
' 3. Gets and modifies the datum target symbol's reference label.
' 4. Examine the Immediate window and graphics area.
'
' NOTE: Because the part document is used elsewhere,
' do not save any changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDatumTargetSym As SldWorks.DatumTargetSym
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()
    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\holecube.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
    warnings)

    'Select face for datum target symbol
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", -7.23565448987529E-03, -2.59480787517532E-02, 0, False, 0, Nothing, 0)

    'Insert datum target symbol
    Set swDatumTargetSym = swModelDocExt.InsertDatumTargetSymbol3("a", "", "", 0, False, 0.003, 0.03, "3", "", True, 12, 0, True,
    False, True, swMoveableDatumStyle_Horizontal)

    'Get and set datum reference label
    Debug.Print ("Current reference label: " & swDatumTargetSym.GetDatumReferenceLabel(0))
    status = swDatumTargetSym.SetDatumReferenceLabel(0, "b")
    Debug.Print ("Modified reference label: " & swDatumTargetSym.GetDatumReferenceLabel(0))
End Sub
```

Insert and Modify Datum Target Symbol Example (VB.NET)

This example shows how to insert and modify a datum target symbol.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Inserts a datum target symbol on the selected
'   face.
' 3. Gets and modifies the datum target symbol's reference label.
' 4. Examine the Immediate window and graphics area.
'
' NOTE: Because the part document is used elsewhere,
' do not save any changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDatumTargetSym As DatumTargetSym
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\holecube.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warn

        'Select face for datum target symbol
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "FACE", -0.00723565448987529, -0.0259480787517532, 0, False, 0, Nothing, 0)

        'Insert datum target symbol
        swDatumTargetSym = swModelDocExt.InsertDatumTargetSymbol3("a", "", "", 0, False, 0.003, 0.03, "3", "", True, 12, 0, True, Fe
        swMoveableDatumStyle_e.swMoveableDatumStyle_Horizontal)

        'Get and set datum reference label
        Debug.Print("Current reference label: " & swDatumTargetSym.GetDatumReferenceLabel(0))
        status = swDatumTargetSym.SetDatumReferenceLabel(0, "b")
        Debug.Print("Modified reference label: " & swDatumTargetSym.GetDatumReferenceLabel(0))

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert and Position DXF/DWG File in Drawing Example (C#)

This example shows how to insert and position a DXF/DWG file in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing.
// 2. Replace DXF_file_path with the pathname of an existing DXF/DWG file.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Inserts the DXF/DWG file as per the specified import data.
// 2. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace InsertDXFDrawing_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            const string sDwgFileName = "DXF_file_path";

            ModelDoc2 swModel = default(ModelDoc2);
            ModelView swModelView = default(ModelView);
            DrawingDoc swDraw = default(DrawingDoc);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            Sketch swSketch = default(Sketch);
            View swView = default(View);
            double[] vPos = null;
            bool bRet = false;
            ImportDxfDwgData importData = default(ImportDxfDwgData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelView = (ModelView)swModel.ActiveView;

            bRet = swModel.Extension.SelectByID2("Sheet1", "SHEET", 0.0, 0.0, 0, false, 0, null, 0);

            swDraw = (DrawingDoc)swModel;
            swFeatMgr = swModel.FeatureManager;
            importData = (ImportDxfDwgData)swApp.GetImportFileData(sDwgFileName);

            // Unit
            importData.set_LengthUnit("", (int)swLengthUnit_e.SWINCHES);

            // Position
            bRet = importData.SetPosition("", (int)swDwgImportEntitiesPositioning_e.swDwgEntitiesCentered, 0, 0);

            // Sheet scale
            bRet = importData.SetSheetScale("", 1.0, 2.0);

            // Paper size
            bRet = importData.SetPaperSize("", (int)swDwgPaperSizes_e.swDwgPaperAsize, 0.0, 0.0);

            //Import method
            importData.set_ImportMethod("", (int)swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingDrawing);

            // Import file with importData
            swFeat = swFeatMgr.InsertDwgOrDxfFile2(sDwgFileName, importData);
            swSketch = (Sketch)swFeat.GetSpecificFeature2();

            swView = (View)swDraw.GetFirstView();

            while ((swView != null))
            {
                if (object.ReferenceEquals(swSketch, swView.GetSketch()))
                {
                    break;
                }
                swView = (View)swView.GetNextView();
            }

            vPos = (double[])swView.Position;

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Sketch = " + swFeat.Name);
            Debug.Print(" View = " + swView.Name);
            Debug.Print(" Old Pos = (" + vPos[0] * 1000.0 + ", " + vPos[1] * 1000.0 + ") mm");

            // Move to right
        }
    }
}
```

```
vPos[0] = vPos[0] + 0.01;
swView.Position = vPos;

vPos = (double[])swView.Position;
Debug.Print("    New Pos    = (" + vPos[0] * 1000.0 + ", " + vPos[1] * 1000.0 + ") mm");

// Redraw
double[] rect = null;
rect = null;
swModelView.GraphicsRedraw(rect);
}

public SldWorks swApp;
}
}
```

Insert and Position DXF/DWG File in Drawing Example (VBA)

This example shows how to insert and position a DXF/DWG file in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing.
' 2. Replace DXF_file_path with the pathname of an existing DXF/DWG file.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts the DXF/DWG file per the specified import data.
' 2. Inspect the Immediate window.
'-----
Option Explicit

Sub main()
    Const sDwgFileName As String = "DXF_file_path"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelView As SldWorks.ModelView
    Dim swDraw As SldWorks.DrawingDoc
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim swView As SldWorks.View
    Dim vPos As Variant
    Dim bRet As Boolean
    Dim importData As SldWorks.ImportDxfDwgData

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelView = swModel.ActiveView

    bRet = swModel.Extension.SelectByID2("Sheet1", "SHEET", 0#, 0#, 0, False, 0, Nothing, 0)
    Set swDraw = swModel
    Set swFeatMgr = swModel.FeatureManager
    Set importData = swApp.GetImportFileData(sDwgFileName)

    ' Unit
    importData.LengthUnit("") = SwConst.swLengthUnit_e.swINCHES

    ' Position
    bRet = importData.SetPosition("", swDwgEntitiesCentered, 0, 0)

    ' Sheet scale
    bRet = importData.SetSheetScale("", 1#, 2#)

    ' Paper size
    bRet = importData.SetPaperSize("", SwConst.swDwgPaperSizes_e.swDwgPaperAsize, 0#, 0#)

    ' Import method
    importData.ImportMethod("") = swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingDrawing

    ' Import file with importData
    Set swFeat = swFeatMgr.InsertDwgOrDxfFile2(sDwgFileName, importData)

    Set swSketch = swFeat.GetSpecificFeature2
    Set swView = swDraw.GetFirstView

    Do While Not swView Is Nothing
        If swSketch Is swView.GetSketch Then
            Exit Do
        End If
        Set swView = swView.GetNextView
    Loop

    vPos = swView.Position

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Sketch = " & swFeat.Name
    Debug.Print " View = " & swView.Name
    Debug.Print " Old Pos = (" & vPos(0) * 1000# & ", " & vPos(1) * 1000# & ") mm"

    ' Move to right
    vPos(0) = vPos(0) + 0.01
    swView.Position = vPos

    vPos = swView.Position
    Debug.Print " New Pos = (" & vPos(0) * 1000# & ", " & vPos(1) * 1000# & ") mm"

```

```
' Redraw
Dim rect() As Double
swModelView.GraphicsRedraw rect
End Sub
```

Insert and Position DXF/DWG File in Drawing Example (VB.NET)

This example shows how to insert and position a DXF/DWG file in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing.
' 2. Replace DXF_file_path with the pathname of an existing DXF/DWG file.
' 3. Open the Immediate window.
'-----
' Postconditions:
' 1. Inserts the DXF/DWG file per the specified import data.
' 2. Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Const sDwgFileName As String = "DXF_file_path"

        Dim swModel As ModelDoc2
        Dim swModelView As ModelView
        Dim swDraw As DrawingDoc
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swSketch As Sketch
        Dim swView As View
        Dim vPos As Object
        Dim bRet As Boolean
        Dim importData As ImportDxfDwgData

        swModel = swApp.ActiveDoc
        swModelView = swModel.ActiveView

        bRet = swModel.Extension.SelectByID2("Sheet1", "SHEET", 0.0#, 0.0#, 0, False, 0, Nothing, 0)

        swDraw = swModel
        swFeatMgr = swModel.FeatureManager
        importData = swApp.GetImportFileData(sDwgFileName)

        ' Unit
        importData.LengthUnit("") = swLengthUnit_e.swINCHES

        ' Position
        bRet = importData.SetPosition("", swDwgImportEntitiesPositioning_e.swDwgEntitiesCentered, 0, 0)

        ' Sheet scale
        bRet = importData.SetSheetScale("", 1.0#, 2.0#)

        ' Paper size
        bRet = importData.SetPaperSize("", swDwgPaperSizes_e.swDwgPaperASize, 0.0#, 0.0#)

        ' Import method
        importData.ImportMethod("") = swImportDxfDwg_ImportMethod_e.swImportDxfDwg_ImportToExistingDrawing

        'Import file with importData
        swFeat = swFeatMgr.InsertDwgOrDxfFile2(sDwgFileName, importData)
        swSketch = swFeat.GetSpecificFeature2

        swView = swDraw.GetFirstView

        Do While Not swView Is Nothing
            If swSketch Is swView.GetSketch Then
                Exit Do
            End If
            swView = swView.GetNextView
        Loop

        vPos = swView.Position

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Sketch = " & swFeat.Name)
        Debug.Print(" View = " & swView.Name)
        Debug.Print(" Old Pos = (" & vPos(0) * 1000.0# & ", " & vPos(1) * 1000.0# & ") mm")

        ' Move to right
        vPos(0) = vPos(0) + 0.01
        swView.Position = vPos

        vPos = swView.Position
        Debug.Print(" New Pos = (" & vPos(0) * 1000.0# & ", " & vPos(1) * 1000.0# & ") mm")

        ' Redraw
        Dim rect() As Double
    
```

```
rect = Nothing  
swModelView.GraphicsRedraw(rect)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Insert and Resize Sketch Slot Example (C#)

This example shows how to insert a sketch slot and resize it.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Open the Immediate window..
//
//Postconditions:
// 1. Creates a sketch and inserts a sketch slot.
// 2. Press F5 after examining the graphics area.
// 3. Resizes the slot.
// 4. Examine the graphics area and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swExt;
        SelectionMgr swSelMgr;
        bool boolstatus;
        PartDoc swPart;
        SketchManager skManager;

        public void Main()
        {
            swModel = swApp.ActiveDoc as ModelDoc2;
            swExt = swModel.Extension;
            swSelMgr = swModel.SelectionManager as SelectionMgr;
            skManager = swModel.SketchManager;
            swPart = swModel as PartDoc;
            boolstatus = swExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);

            skManager.InsertSketch(true);
            SketchSlot swSketchSlot;
            swSketchSlot = skManager.CreateSketchSlot((int)swSketchSlotCreationType_e.swSketchSlotCreationType_line,
(int)swSketchSlotLengthType_e.swSketchSlotLengthType_CenterCenter,
            0.05, -0.05, 0, 0, 0.05, 0, 0, 0, 0, 1, false);
            swSketchSlotLengthType_e lengthType;
            lengthType = (swSketchSlotLengthType_e)swSketchSlot.LengthType;
            Debug.Print("Length: " + swSketchSlot.Length);
            Debug.Print("Length Type: " + lengthType.ToString());
            Debug.Print("Width: " + swSketchSlot.Width);

            System.Diagnostics.Debugger.Break();

            // Edit the original slot
            swSketchSlot.Width = swSketchSlot.Width * 2.0;
            Debug.Print("Modified width: " + swSketchSlot.Width);
            skManager.InsertSketch(true);
        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Insert and Resize Sketch Slot Example (VBA)

This example shows how to insert and resize a sketch slot.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window..
'
' Postconditions:
' 1. Creates a sketch and inserts a sketch slot.
' 2. Press F5 after examining the graphics area.
' 3. Resizes the slot.
' 4. Examine the graphics area and Immediate window.
'-----
Option Explicit
```

```
Dim swApp As SldWorks.SldWorks
Dim swModel As ModelDoc2
Dim swExt As ModelDocExtension
Dim swSelMgr As SelectionMgr
Dim boolstatus As Boolean
Dim swPart As PartDoc
Dim skManager As SketchManager

Sub main()

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
Set skManager = swModel.SketchManager
Set swPart = swModel

boolstatus = swExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

skManager.InsertSketch (True)
Dim swSketchSlot As SketchSlot
' Insert a sketch slot
Set swSketchSlot = skManager.CreateSketchSlot(swSketchSlotCreationType_e.swSketchSlotCreationType_line,
swSketchSlotLengthType_e.swSketchSlotLengthType_CenterCenter, 0.05, -0.05, 0, 0, 0.05, 0, 0, 0, 0, 0, 1, False)
Dim lengthType As swSketchSlotLengthType_e
lengthType = swSketchSlot.lengthType
Debug.Print "Length: " & swSketchSlot.Length
Debug.Print "Length Type: " & lengthType
Debug.Print "Width: " & swSketchSlot.Width

Stop

' Edit the slot
swSketchSlot.Width = swSketchSlot.Width * 2#
Debug.Print "Modified width: " & swSketchSlot.Width
skManager.InsertSketch (True)

End Sub
```

Insert and Resize Sketch Slot Example (VB.NET)

This example shows how to insert a sketch slot and resize it.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window..
'
' Postconditions:
' 1. Creates a sketch and inserts a sketch slot.
' 2. Press F5 after examining the graphics area.
' 3. Resizes the slot.
' 4. Examine the graphics area and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean
    Dim swPart As PartDoc
    Dim skManager As SketchManager

    Public Sub main()

        swModel = swApp.ActiveDoc
        swExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        skManager = swModel.SketchManager
        swPart = swModel

        boolstatus = swExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

        skManager.InsertSketch(True)

        Dim swSketchSlot As SketchSlot
        swSketchSlot = skManager.CreateSketchSlot(swSketchSlotCreationType_e.swSketchSlotCreationType_line,
        swSketchSlotLengthType_e.swSketchSlotLengthType_CenterCenter, 0.05, -0.05, 0, 0, 0.05, 0, 0, 0, 0, 0, 1, False)

        Dim lengthType As swSketchSlotLengthType_e
        lengthType = swSketchSlot.LengthType

        Debug.Print("Length: " & swSketchSlot.Length)
        Debug.Print("Length Type: " & lengthType.ToString())
        Debug.Print("Width: " & swSketchSlot.Width)

        Stop

        ' Edit the original slot
        swSketchSlot.Width = swSketchSlot.Width * 2.0#
        Debug.Print "Modified width: " & swSketchSlot.Width
        skManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
End Class
```

Insert and Rotate Camera Example (C#)

This example shows how to insert and rotate a camera.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a camera in the model, changes its type to floating,
//    and rotates it (i.e., modifies its pitch and yaw).
// 2. Examine the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace PitchYawCameraCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Camera swCamera = default(Camera);
            int fileerror = 0;
            int filewarning = 0;
            bool boolstatus = false;

            // Open part document
            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\coffee.cup.sldprt",
(int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref fileerror, ref filewarning);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Insert a camera
            swCamera = (Camera)swModelDocExt.InsertCamera();

            // Set camera type to floating
            swCamera.Type = (int)swCameraType_e.swCameraType_Floating;

            // Show camera
            boolstatus = swModelDocExt.SelectByID2("Camera1", "CAMERAS", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModel.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDisplayCameras, true);
            swModel.GraphicsRedraw2();

            // Get camera's pitch and yaw settings
            // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print("Original pitch (up or down angle) = " + swCamera.Pitch * 57.3 + " deg");
            Debug.Print("Original yaw (side-to-side angle) = " + swCamera.Yaw * 57.3 + " deg");
            Debug.Print(" ");

            // Rotate camera
            swCamera.Pitch = -25;
            swCamera.Yaw = 150;

            // New pitch and yaw settings
            Debug.Print("New pitch (up or down angle) = " + swCamera.Pitch * 57.3 + " deg");
            Debug.Print("New yaw (side-to-side angle) = " + swCamera.Yaw * 57.3 + " deg");
            swModel.GraphicsRedraw2();
        }
    }
}
```

```
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

}

}
```

Insert and Rotate Camera Example (VBA)

This example shows how to insert a camera and rotate a camera.

```
'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a camera in the model, changes its type to floating,
'    and rotates it (i.e., modifies its pitch and yaw).
' 2. Examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel As SldWorks.ModelDoc2
```

```
Dim swModelDocExt As SldWorks.ModelDocExtension
```

```
Dim swCamera As SldWorks.Camera
```

```
Dim fileerror As Long, filewarning As Long
```

```
Dim boolstatus As Boolean
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
```

```
    ' Open part document
```

```
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\coffeecup.sldprt", swDocPART,
    swOpenDocOptions_Silent, "", fileerror, filewarning
```

```
    Set swModel = swApp.ActiveDoc
```

```
    Set swModelDocExt = swModel.Extension
```

```
    ' Insert a camera
```

```
    Set swCamera = swModelDocExt.InsertCamera
```

```
    ' Set camera type to floating
```

```
    swCamera.Type = swCameraType_Floating
```

```
    ' Show camera
```

```
    boolstatus = swModelDocExt.SelectByID2("Camera1", "CAMERAS", 0, 0, 0, False, 0, Nothing, 0)
```

```
    boolstatus = swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayCameras, True)
```

```
    swModel.GraphicsRedraw2
```

```
    ' Get camera's pitch and yaw settings
```

```
    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
```

```
    Debug.Print "Original pitch (up or down angle)      = " & swCamera.Pitch * 57.3 & " deg"
```

```
    Debug.Print "Original yaw (side-to-side angle)      = " & swCamera.Yaw * 57.3 & " deg"
```

```
    Debug.Print " "
```

```
    ' Rotate camera
```

```
    swCamera.Pitch = -25
```

```
    swCamera.Yaw = 150
```

```
    ' New pitch and yaw settings
```

```
    Debug.Print "New pitch (up or down angle)          = " & swCamera.Pitch * 57.3 & " deg"
```

```
    Debug.Print "New yaw (side-to-side angle)          = " & swCamera.Yaw * 57.3 & " deg"
```

```
    swModel.GraphicsRedraw2
```

```
End Sub
```

Insert and Rotate Camera Example (VB.NET)

This example shows how to insert and rotate a camera.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a camera in the model, changes its type to floating,
'    and rotates it (i.e., modifies its pitch and yaw).
' 2. Examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swCamera As Camera
        Dim fileerror As Long, filewarning As Long
        Dim boolstatus As Boolean

        ' Open part document
        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\coffee\cup.sldprt",
        swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", fileerror, filewarning)
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Insert a camera
        swCamera = swModelDocExt.InsertCamera

        ' Set camera type to floating
        swCamera.Type = swCameraType_e.swCameraType_Floating

        ' Show camera
        boolstatus = swModelDocExt.SelectByID2("Camera1", "CAMERAS", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayCameras, True)
        swModel.GraphicsRedraw2()

        ' Get camera's pitch and yaw settings
        Debug.Print("Original pitch (up or down angle)    = " & swCamera.Pitch * 57.3 & " deg")
        Debug.Print("Original yaw (side-to-side angle)    = " & swCamera.Yaw * 57.3 & " deg")
        Debug.Print(" ")

        ' Rotate camera
        swCamera.Pitch = -25
        swCamera.Yaw = 150

        ' New pitch and yaw settings
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print("New pitch (up or down angle)                = " & swCamera.Pitch * 57.3 & " deg")
        Debug.Print("New yaw (side-to-side angle)                    = " & swCamera.Yaw * 57.3 & " deg")
        swModel.GraphicsRedraw2()

    End Sub

```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Insert and Save Virtual Assembly Example (C#)

This example shows how to create and save a virtual sub-assembly.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Rename the namespace of this macro to match your C# project's name.
// 3. Open an Immediate Window.
// 4. Run this macro.
//
// Postconditions:
// 1. Tools > Options > System Options > Assemblies >
//    Save new components to external files is selected,
//    and InsertNewAssembly is called, passing in FileName
//    to save the sub-assembly:
//    a. MyTestValveAssembly<1> displays in the FeatureManager design tree.
//    b. MyTestValveAssembly.sldasm is saved in the assembly's directory.
// 2. Next, Tools > Options > System Options > Assemblies >
//    Save new components to external files is de-selected,
//    and InsertNewAssembly is called, passing in FileName
//    to save the sub-assembly.
//    a. A virtual sub-assembly displays in the FeatureManager design tree.
//    b. The FileName parameter is ignored, and the virtual sub-assembly
//       is not saved.
// 3. The Immediate Window displays the error codes
//    as defined in swInsertNewAssemblyErrorCode_e.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace InsertNewAssembly_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        AssemblyDoc swAssy;
        string tmpPath;

        public void Main()
        {
            // Turn on Tools > Options > System Options > Assemblies > Save new components to external files
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, true);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            string strCompModelName = null;
            strCompModelName = "MyTestValveAssembly.sldasm";

            // Save the new sub-assembly to the same folder where the parent assembly resides
            tmpPath = swModel.GetPathName();
            string[] tok;
            tok = tmpPath.Split('\\');

            // reconstruct the assembly path without the file name
            int i;
            string virAssPath = "";
            for (i = 0; i < tok.Length - 1; i++)
            {
                virAssPath = virAssPath + tok[i] + "\\";
            }

            Debug.Print(virAssPath);
            swAssy = (AssemblyDoc)swModel;

            // Create a virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
            Debug.Print("First virtual sub-assembly created and saved? " + swAssy.InsertNewAssembly(virAssPath +
strCompModelName));

            // Turn off Tools > Options > System Options > Assemblies > Save new components to external files
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, false);

            // Create another virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
            Debug.Print("Second virtual sub-assembly created but not saved? " + swAssy.InsertNewAssembly(virAssPath +
strCompModelName));
        }
    }

    public SldWorks swApp;
```


Insert and Save Virtual Assembly Example (VBA)

This example shows how to create and save a virtual sub-assembly.

```
'-----
' Preconditions:
'
' 1. Open an assembly document.
' 2. Open an Immediate Window.
' 3. Run this macro.
'
' Postconditions:
'
' 1. First, Tools > Options > System Options > Assemblies > Save new components
'    to external files is selected, and InsertNewAssembly is called,
'    passing in FileName to save the sub-assembly:
'    a. MyTestValveAssembly<1> displays in the FeatureManager design tree.
'    b. MyTestValveAssembly.sldasm is saved in the assembly's directory.
' 2. Next, Tools > Options > System Options > Assemblies > Save new components
'    to external files is de-selected, and InsertNewAssembly is called,
'    passing in FileName to save the sub-assembly.
'    a. A virtual sub-assembly displays in the FeatureManager design tree.
'    b. The FileName parameter is ignored, and the virtual sub-assembly
'       is not saved.
' 3. The Immediate Window displays the error codes as defined in
'    swInsertNewAssemblyErrorCode_e.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim tmpPath As String
Dim boolstat As Boolean
Dim longstatus As Long
Dim longwarnings As Long

Sub Main()
    Set swApp = Application.SldWorks

    ' Turn on Tools > Options > System Options > Assemblies > Save new components to external files
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, True

    Set swModel = swApp.ActiveDoc
    boolstat = True
    Dim strCompModelname As String
    strCompModelname = "MyTestValveAssembly.sldasm"

    ' Save the new assembly in the same folder as the original assembly
    tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))
    Set swAssy = swModel

    ' Create a virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
    Debug.Print "First virtual sub-assembly created and saved? " & swAssy.InsertNewAssembly(tmpPath + strCompModelname)

    ' Turn off Tools > Options > System Options > Assemblies > Save new components to external files
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, False

    ' Create another virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
    Debug.Print "Second virtual sub-assembly created but not saved? " & swAssy.InsertNewAssembly(tmpPath + strCompModelname)
End Sub
```

Insert and Save Virtual Assembly Example (VB.NET)

This example shows how to create and save a virtual sub-assembly.

```

'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Open an Immediate Window.
' 3. Run this macro.
'
' Postconditions:
' 1. First, Tools > Options > System Options > Assemblies > Save new components
'    to external files is selected, and InsertNewAssembly is called, passing in
'    FileName to save the sub-assembly:
'    a. MyTestValveAssembly<1> displays in the FeatureManager design tree.
'    b. MyTestValveAssembly.sldasm is saved in the assembly's directory.
' 2. Next, Tools > Options > System Options > Assemblies > Save new components
'    to external files is de-selected, and InsertNewAssembly is called,
'    passing in FileName to save the sub-assembly.
'    a. A virtual sub-assembly displays in the FeatureManager design tree.
'    b. The FileName parameter is ignored, and the virtual sub-assembly
'       is not saved.
' 3. The Immediate Window displays the error codes
'    as defined in swInsertNewAssemblyErrorCode_e.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim tmpPath As String
    Dim boolstat As Boolean
    Dim longstatus As Long
    Dim longwarnings As Long

    Sub Main()

        ' Turn on Tools > Options > System Options > Assemblies > Save new components to external files
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, True)

        swModel = swApp.ActiveDoc
        boolstat = True
        Dim strCompModelname As String
        strCompModelname = "MyTestValveAssembly.sldasm"

        ' Save the new assembly in the same folder as the original assembly
        tmpPath = Left(swModel.GetPathName, InStrRev(swModel.GetPathName, "\"))
        swAssy = swModel

        ' Create a virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
        Debug.Print("First virtual sub-assembly created and saved? " & swAssy.InsertNewAssembly(tmpPath + strCompModelname))

        ' Turn off Tools > Options > System Options > Assemblies > Save new components to external files
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSaveNewComponentsToExternalFile, False)

        ' Create another virtual sub-assembly and print the error code as defined in swInsertNewAssemblyErrorCode_e
        Debug.Print("Second virtual sub-assembly created but not saved? " & swAssy.InsertNewAssembly(tmpPath + strCompModelname))
    End Sub

    Public swApp As SldWorks

End Class

```

Insert and Show BOM Table and BOM Balloon Example (C#)

This example shows how to insert a Bill of Materials (BOM) table and balloon in a drawing document.

```
//-----
// Preconditions:
// 1. Specified file and template exist.
// 2. Open an Immediate Window.
//
// Postconditions:
// 1. Inserts an indented BOM table.
// 2. Inserts a BOM balloon annotation.
// 3. Inspect the Immediate Window for the BOM feature properties.
//
// NOTE: Because the drawing is used elsewhere, do not save any changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace IViewInsertBomTable4CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            View swView = default(View);
            BomTableAnnotation swBOMAnnotation = default(BomTableAnnotation);
            BomFeature swBOMFeature = default(BomFeature);
            Note swNote = default(Note);
            BalloonOptions BomBalloonParams = default(BalloonOptions);
            bool boolstatus = false;
            int AnchorType = 0;
            int NbrType = 0;
            int BomType = 0;
            int nErrors = 0;
            int nWarnings = 0;
            string Configuration = null;
            string TableTemplate = null;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw", (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDoc
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            boolstatus = swDrawing.ActivateView("Drawing View1");
            swView = (View)swDrawing.ActiveDrawingView;

            // Insert indented BOM table
            AnchorType = (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft;
            BomType = (int)swBomType_e.swBomType_Indented;
            NbrType = (int)swNumberingType_e.swNumberingType_Detailed;
            TableTemplate = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-standard.sldbomtbt";
            Configuration = "";
            swBOMAnnotation = (BomTableAnnotation)swView.InsertBomTable6(false, 0.4, 0.3, AnchorType, BomType, Configuration, Table
            true, true);
            swBOMFeature = (BomFeature)swBOMAnnotation.BomFeature;

            Debug.Print("Type of BOM table as defined in swBomType_e: " + (int)swBOMFeature.TableType);
            Debug.Print("Numbering type of BOM table as defined in swNumberingType_e: " + (int)swBOMFeature.NumberingTypeOnIndentedBOM);
            Debug.Print("Value to display when a value is 0 as defined in swZeroQuantityDisplay_e: " +
            (int)swBOMFeature.ZeroQuantityDisplay);
            Debug.Print("Name of configuration used for BOM table: " + swBOMFeature.Configuration);
            Debug.Print("Display as one item? " + swBOMFeature.DisplayAsOneItem);
            Debug.Print("Sequence start number: " + swBOMFeature.SequenceStartNumber);
            Debug.Print("Keep missing items? " + swBOMFeature.KeepMissingItems);
            Debug.Print("Strikeout missing items? " + swBOMFeature.StrikeoutMissingItems);
            Debug.Print(" Replace missing components as defined in swKeepReplacedCompOption_e: " + swBOMFeature.KeepReplacedCompOption);
            Debug.Print("Keep current item numbers when reordering rows? " + swBOMFeature.KeepCurrentItemNumbers);

            boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.1205506330468, 0.261655309417, -0.0004000000000133, false, 0, null,

            BomBalloonParams = swModelDocExt.CreateBalloonOptions();
            BomBalloonParams.Style = (int)swBalloonStyle_e.swBS_Circular;
            BomBalloonParams.Size = (int)swBalloonFit_e.swBF_2Chars;
            BomBalloonParams.UpperTextContent = (int)swBalloonTextContent_e.swBalloonTextItemNumber;
            BomBalloonParams.ShowQuantity = true;
            BomBalloonParams.QuantityPlacement = (int)swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Right;
            BomBalloonParams.QuantityDenotationText = "PLACES";
            BomBalloonParams.QuantityOverride = false;
            BomBalloonParams.QuantityOverrideValue = "";
            BomBalloonParams.ItemNumberStart = 1;
        }
    }
}
```

```
BomBalloonParams.ItemNumberIncrement = 1;
BomBalloonParams.ItemOrder = (int)swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers;

swNote = (Note)swModelDocExt.InsertBOMBalloon2(BomBalloonParams);

if (swNote.IsBomBalloon())
{
    Debug.Print("Name of BOM balloon: " + swNote.GetName());
}

swDrawing.ForceRebuild();
}

public SldWorks swApp;
}
}
```

Insert and Show BOM Table and BOM Balloon Example (VBA)

This example shows how to insert a Bill of Materials (BOM) table and balloon in a drawing document.

```

'-----
' Preconditions:
' 1. Specified file and template exist.
' 2. Open an Immediate Window.
'
' Postconditions:
' 1. Inserts an indented BOM table.
' 2. Inserts a BOM balloon annotation.
' 3. Inspect the Immediate Window for the BOM feature properties.
'
' NOTE: Because the drawing is used elsewhere, do not save any changes.
'-----

Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swModelDocExt        As SldWorks.ModelDocExtension
Dim swDrawing            As SldWorks.DrawingDoc
Dim swView               As SldWorks.View
Dim swBOMAnnotation      As SldWorks.BomTableAnnotation
Dim swBOMFeature         As SldWorks.BomFeature
Dim swNote               As SldWorks.Note
Dim BomBalloonParams     As SldWorks.BalloonOptions
Dim boolstatus           As Boolean
Dim AnchorType           As Long
Dim BomType              As Long
Dim nErrors              As Long
Dim nWarnings            As Long
Dim Configuration        As String
Dim TableTemplate        As String

Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocDRAWING, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    boolstatus = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView

    ' Insert indented BOM table
    AnchorType = swBOMConfigurationAnchor_TopLeft
    BomType = swBomType_Indented
    TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    Configuration = ""
    Set swBOMAnnotation = swView.InsertBOMTable6(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate, False,
swNumberingType_Detailed, True, True, True)
    Set swBOMFeature = swBOMAnnotation.BomFeature

    Debug.Print "Type of BOM table as defined in swBomType_e: " & swBOMFeature.TableType
    Debug.Print "Numbering type of BOM table as defined in swNumberingType_e: " & swBOMFeature.NumberingTypeOnIndentedBOM
    Debug.Print "Value to display when a value is 0 as defined in swZeroQuantityDisplay_e: " & swBOMFeature.ZeroQuantityDisplay
    Debug.Print "Name of configuration used for BOM table: " & swBOMFeature.Configuration
    Debug.Print "Display as one item? " & swBOMFeature.DisplayAsOneItem
    Debug.Print "Sequence start number: " & swBOMFeature.SequenceStartNumber
    Debug.Print "Keep missing items? " & swBOMFeature.KeepMissingItems
    Debug.Print "Strikeout missing items? " & swBOMFeature.StrikeoutMissingItems
    Debug.Print "Replace missing components as defined in swKeepReplacedCompOption_e: " & swBOMFeature.KeepReplacedCompOption
    Debug.Print "Keep current item numbers when reordering rows? " & swBOMFeature.KeepCurrentItemNumbers

    boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.1205506330468, 0.261655309417, -4.000000000133E-04, False, 0, Nothing, 0)

    Set BomBalloonParams = swModel.Extension.CreateBalloonOptions()
    BomBalloonParams.Style = swBS_Circular
    BomBalloonParams.Size = swBF_2Chars
    BomBalloonParams.UpperTextContent = swBalloonTextItemNumber
    BomBalloonParams.ShowQuantity = True
    BomBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_Right
    BomBalloonParams.QuantityDenotationText = "PLACES"
    BomBalloonParams.QuantityOverride = False
    BomBalloonParams.QuantityOverrideValue = ""
    BomBalloonParams.ItemNumberStart = 1
    BomBalloonParams.ItemNumberIncrement = 1
    BomBalloonParams.ItemOrder = swBalloonItemNumbers_DoNotChangeItemNumbers

    Set swNote = swModelDocExt.InsertBOMBalloon2(BomBalloonParams)

    ' Get whether balloon is a BOM balloon
    ' If so, print the name of the BOM balloon

    If swNote.IsBomBalloon Then
        Debug.Print ("Name of BOM balloon: " & swNote.GetName)
    End If

    swDrawing.ForceRebuild
End Sub

```

Insert and Show BOM Table and BOM Balloon Example (VB.NET)

This example shows how to insert a Bill of Materials (BOM) table and balloon in a drawing document.

```

-----
Preconditions:
1. Specified file and template exist.
2. Open an Immediate Window.

Postconditions:
1. Inserts an indented BOM table.
2. Inserts a BOM balloon annotation.
3. Inspect the Immediate Window for the BOM feature properties.

NOTE: Because the drawing is used elsewhere, do not
save any changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim swView As View
        Dim swBOMAnnotation As BomTableAnnotation
        Dim swBOMFeature As BomFeature
        Dim swNote As Note
        Dim BomBalloonParams As BalloonOptions
        Dim boolstatus As Boolean
        Dim AnchorType As Long
        Dim BomType As Integer
        Dim nErrors As Integer
        Dim nWarnings As Integer
        Dim Configuration As String
        Dim TableTemplate As String
        Dim NbrType As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", nErrors, nWarning
s)

        swDrawing = swModel
        swModelDocExt = swModel.Extension
        boolstatus = swDrawing.ActivateView("Drawing View1")
        swView = swDrawing.ActiveDrawingView

        ' Insert indented BOM table
        AnchorType = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft
        BomType = swBomType_e.swBomType_Indented
        TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
        Configuration = ""
        NbrType = swNumberingType_e.swNumberingType_Detailed
        swBOMAnnotation = swView.InsertBOMTable6(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate, False, NbrType, True, True, True)
        swBOMFeature = swBOMAnnotation.BomFeature

        Debug.Print("Type of BOM table as defined in swBomType_e: " & swBOMFeature.TableType)
        Debug.Print("Numbering type of BOM table as defined in swNumberingType_e: " & swBOMFeature.NumberingTypeOnIndentedBOM)
        Debug.Print("Value to display when a value is 0 as defined in swZeroQuantityDisplay_e: " & swBOMFeature.ZeroQuantityDisplay)
        Debug.Print("Name of configuration used for BOM table: " & swBOMFeature.Configuration)
        Debug.Print("Display as one item? " & swBOMFeature.DisplayAsOneItem)
        Debug.Print("Sequence start number: " & swBOMFeature.SequenceStartNumber)
        Debug.Print("Keep missing items? " & swBOMFeature.KeepMissingItems)
        Debug.Print("Strikeout missing items? " & swBOMFeature.StrikeoutMissingItems)
        Debug.Print(" Replace missing components as defined in swKeepReplacedCompOption_e: " & swBOMFeature.KeepReplacedCompOption)
        Debug.Print("Keep current item numbers when reordering rows? " & swBOMFeature.KeepCurrentItemNumbers)

        boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.1205506330468, 0.261655309417, -0.0004000000000133, False, 0, Nothing, 0)

        BomBalloonParams = swModel.Extension.CreateBalloonOptions()
        BomBalloonParams.Style = swBalloonStyle_e.swBS_Circular
        BomBalloonParams.Size = swBalloonFit_e.swBF_2Chars
        BomBalloonParams.UpperTextContent = swBalloonTextContent_e.swBalloonTextItemNumber
        BomBalloonParams.ShowQuantity = True
        BomBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Right
        BomBalloonParams.QuantityDenotationText = "PLACES"
        BomBalloonParams.QuantityOverride = False
        BomBalloonParams.QuantityOverrideValue = ""
        BomBalloonParams.ItemNumberStart = 1
        BomBalloonParams.ItemNumberIncrement = 1
        BomBalloonParams.ItemOrder = swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers

        swNote = swModelDocExt.InsertBOMBalloon2(BomBalloonParams)

        ' Get whether balloon is a BOM balloon
        ' If so, print the name of the BOM balloon

        If swNote.IsBomBalloon Then
            Debug.Print("Name of BOM balloon: " & swNote.GetName)
        End If

        swDrawing.ForceRebuild()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert and Show BOM Table in Assembly Example (C#)

This example shows how to insert and show a BOM table in an assembly document.

```
//-----
// Preconditions:
// 1. Verify that the specified file to open and template exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts an indented BOM table.
// 2. Inserts a split-circle stacked balloon, which uses the
//    BOM table item number for its upper text,
//    at the selected face.
// 3. Examine the BOM table, stacked balloon, and Immediate
//    window.
//
// NOTE: Because this assembly document is used elsewhere,
// do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace IBomFeatureInsertBomTable3CSharp.csproj
{
    partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            BomTableAnnotation swBOMAnnotation = default(BomTableAnnotation);
            BomFeature swBOMFeature = default(BomFeature);
            StackedBalloonOptions StackedBalloonParams = default(StackedBalloonOptions);
            Note swNote = default(Note);
            bool boolstatus = false;
            int BomType = 0;
            int nbrType = 0;
            string Configuration = null;
            string TemplateName = null;
            int nErrors = 0;
            int nWarnings = 0;

            // Open assembly document
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\arm2.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref nErrors, ref nW
arnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Insert BOM table
            TemplateName = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-standard.sldbomtbt";
            BomType = (int)swBomType_e.swBomType_Indented;
            Configuration = "Default";
            nbrType = (int)swNumberingType_e.swNumberingType_Detailed;

            swBOMAnnotation = (BomTableAnnotation)swModelDocExt.InsertBomTable4(TemplateName, 0, 0, BomType, Configuration, false, nbrType, true, true);
            swBOMFeature = (BomFeature)swBOMAnnotation.BomFeature;

            // Print the name of the configuration used for the BOM table
            Debug.Print("Name of configuration used for BOM table: " + swBOMFeature.Configuration);

            // Insert BOM balloon for the selected face
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.091853347996107, -0.0104709589619745, 0.00174830255600256, false, 0, null, 0);

            StackedBalloonParams = (StackedBalloonOptions)swModelDocExt.CreateStackedBalloonOptions();
            StackedBalloonParams.BalloonsPerLine = 10;
            StackedBalloonParams.StackDirection = (int)swStackedBalloonDirection_e.swStackedBalloonDir_Right;
            StackedBalloonParams.Style = (int)swBalloonStyle_e.swBS_SplitCirc;
            StackedBalloonParams.LowerTextContent = (int)swBalloonTextContent_e.swBalloonTextCustom;
            StackedBalloonParams.LowerText = "Lower Text";
            StackedBalloonParams.ShowQuantity = true;
            StackedBalloonParams.Size = (int)swBalloonFit_e.swBF_Tightest;
            StackedBalloonParams.QuantityPlacement = (int)swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top;
            StackedBalloonParams.QuantityDenotationText = "Denotation Text";
            StackedBalloonParams.QuantityOverride = false;
            StackedBalloonParams.ItemNumberStart = 1;
            StackedBalloonParams.ItemNumberIncrement = 1;
            StackedBalloonParams.ItemOrder = (int)swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers;

            swNote = (Note)swModelDocExt.InsertStackedBalloon2(StackedBalloonParams);

            swModel.ViewZoomtoFit2();

            // Get whether balloon is a stacked balloon;
            // if so, print the name of the balloon

            if (swNote.IsStackedBalloon())
            {
                Debug.Print("Name of stacked balloon: " + swNote.GetName());
            }
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
} }
```

Insert and Show BOM Table in Assembly Example (VBA)

This example shows how to insert and show a BOM table in an assembly document.

```

'-----
' Preconditions:
' 1. Verify that the specified file and template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts an indented BOM table.
' 2. Inserts a split-circle stacked balloon, which uses the
'    BOM table item number for its upper text,
'    at the selected face.
' 3. Examine the BOM table, stacked balloon, and Immediate
'    window.
'
' NOTE: Because this assembly document is used elsewhere,
' do not save changes.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModelDocExt        As SldWorks.ModelDocExtension
    Dim swBOMAnnotation      As SldWorks.BomTableAnnotation
    Dim swBOMFeature         As SldWorks.BomFeature
    Dim swNote               As SldWorks.Note
    Dim boolstatus           As Boolean
    Dim BomType              As Long
    Dim Configuration        As String
    Dim TemplateName         As String
    Dim nErrors              As Long
    Dim nWarnings            As Long

    Set swApp = Application.SldWorks

    ' Open assembly document
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\arm2.sldasm",
    swDocASSEMBLY, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    Set swModelDocExt = swModel.Extension

    ' Insert BOM table
    TemplateName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    BomType = swBomType_Indented
    Configuration = "Default"
    Set swBOMAnnotation = swModelDocExt.InsertBomTable4(TemplateName, 0, 1, BomType, Configuration, False,
    swNumberingType_Detailed, True, True)
    Set swBOMFeature = swBOMAnnotation.BomFeature

    ' Print the name of the configuration used for the BOM table
    Debug.Print "Name of configuration used for BOM table: " & swBOMFeature.Configuration

    ' Insert BOM balloon for the selected face, which
    ' belongs to the part Side
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.091853347996107, -1.04709589619745E-02, 1.74830255600256E-03, False, 0,
    Nothing, 0)

    Dim StackedBalloonParams as SldWorks.StackedBalloonOptions
    Set StackedBalloonParams = swModelDocExt.CreateStackedBalloonOptions()
    StackedBalloonParams.BalloonsPerLine = 10
    StackedBalloonParams.StackDirection = swStackedBalloonDir_Right
    StackedBalloonParams.Style = swBS_SplitCirc
    StackedBalloonParams.Size = swBF_Tightest
    StackedBalloonParams.UpperTextContent = swBalloonTextItemNumber
    StackedBalloonParams.LowerTextContent = swBalloonTextCustom

```

```
StackedBalloonParams.LowerText = "Lower text"  
StackedBalloonParams.ShowQuantity = True  
StackedBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_Top  
StackedBalloonParams.QuantityDenotationText = "PLACES"  
StackedBalloonParams.QuantityOverride = False  
StackedBalloonParams.ItemNumberStart = 1  
StackedBalloonParams.ItemNumberIncrement = 1  
StackedBalloonParams.ItemOrder = swBalloonItemNumbers_DoNotChangeItemNumbers
```

```
Set swNote = swModelDocExt.InsertStackedBalloon2(StackedBalloonParams)
```

```
swModel.ViewZoomtofit2
```

```
' Get whether balloon is a stacked balloon;
```

```
' if so, print the name of the balloon
```

```
If swNote.IsStackedBalloon Then
```

```
    Debug.Print ("Name of stacked balloon: " & swNote.GetName)
```

```
End If
```

```
End Sub
```

Insert and Show BOM Table in Assembly Example (VB.NET)

This example shows how to insert and show a BOM table in an assembly document.

```

-----
Preconditions:
1. Verify that the specified file and template exist.
2. Open the Immediate window.

Postconditions:
1. Inserts an indented BOM table.
2. Inserts a split-circle stacked balloon, which uses the
BOM table item number for its upper text,
at the selected face.
3. Examine the BOM table, stacked balloon, and Immediate
window.

NOTE: Because this assembly document is used elsewhere,
do not save changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swBOMAnnotation As BomTableAnnotation
        Dim swBOMFeature As BomFeature
        Dim StackedBalloonParams As StackedBalloonOptions
        Dim swNote As Note
        Dim boolstatus As Boolean
        Dim BomType As Integer
        Dim nbrType As Integer
        Dim Configuration As String
        Dim TemplateName As String
        Dim nErrors As Integer
        Dim nWarnings As Integer

        ' Open assembly document
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\arm2.sldasm", swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", nErrors, nWarnings)
        swModelDocExt = swModel.Extension

        ' Insert BOM table
        TemplateName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
        BomType = swBomType_e.swBomType_Indented
        Configuration = "Default"
        nbrType = swNumberingType_e.swNumberingType_Detailed

        swBOMAnnotation = swModelDocExt.InsertBomTable4(TemplateName, 0, 0, BomType, Configuration, False, nbrType, True, True)
        swBOMFeature = swBOMAnnotation.BomFeature

        ' Print the name of the configuration used for the BOM table
        Debug.Print("Name of configuration used for BOM table: " & swBOMFeature.Configuration)

        ' Insert BOM balloon for the selected face
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.091853347996107, -0.0104709589619745, 0.00174830255600256, False, 0, Nothing, 0)

        StackedBalloonParams = swModelDocExt.CreateStackedBalloonOptions()
        StackedBalloonParams.BalloonsPerLine = 10
        StackedBalloonParams.StackDirection = swStackedBalloonDirection_e.swStackedBalloonDir_Right
        StackedBalloonParams.Style = swBalloonStyle_e.swBS_SplitCirc
        StackedBalloonParams.LowerTextContent = swBalloonTextContent_e.swBalloonTextCustom
        StackedBalloonParams.LowerText = "Lower Text"
        StackedBalloonParams.ShowQuantity = True
        StackedBalloonParams.Size = swBalloonFit_e.swBF_Tightest
        StackedBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top
        StackedBalloonParams.QuantityDenotationText = "Denotation Text"
        StackedBalloonParams.QuantityOverride = False
        StackedBalloonParams.ItemNumberStart = 1
        StackedBalloonParams.ItemNumberIncrement = 1
        StackedBalloonParams.ItemOrder = swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers

        swNote = swModelDocExt.InsertStackedBalloon2(StackedBalloonParams)

        swModel.ViewZoomtofit2()

        ' Get whether balloon is a stacked balloon;
        ' if so, print the name of the balloon
        If swNote.IsStackedBalloon Then

            Debug.Print("Name of stacked balloon: " & swNote.GetName)

        End If

    End Sub

End Class

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

Insert and Use Plane with Manipulator Example (C#)

This example shows how to insert a plane with a manipulator.

```
//-----
// Preconditions:
// 1. In the IDE:
//    a. Click Project > Add Reference > SolidWorks.Interop.swpublished
//       on the .NET tab.
//    b. Click Project > Add Reference > Microsoft.VisualBasic
//       on the .NET tab.
//    c. Copy the code in SolidWorksMacro.cs to your project
//       and create a class and copy the code in Class1.cs
//       to that class.
//    d. Open the Immediate window.
// 2. Verify that the specified part exists.
//
// Postconditions:
// 1. Opens the part.
// 2. Displays a plane with manipulator.
// 3. Gets the distance, angles, height, and width of the plane.
// 4. Examine the Immediate window.
// 5. Hold down the left-mouse button and drag the
//    plane up and down using the manipulator, which calls the
//    handler. Gets the handle index at each drag.
// 6. Click an edge of the plane and hold down the left-mouse button
//    and rotate the plane, which calls the handler. Gets the handle
//    index at each rotation.
// 7. Examine the Immediate window.
//
// NOTE: Because the part document is used elsewhere, do not
// save changes.
//-----

// SolidWorksMacro.cs
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using SolidWorks.Interop.swpublished;
using Microsoft.VisualBasic;

namespace PlaneManipulatorCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public Class1 swHdlr;
        public Manipulator swManipulator;
        public PlaneManipulator swPlaneManipulator;

        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            ModelViewManager swModelViewMgr = default(ModelViewManager);

            string fileName = null;
            int errors = 0;
            int warnings = 0;

            swHdlr = new Class1();

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            // Create the plane with a manipulator
            swModelViewMgr = (ModelViewManager)swModelDoc.ModelViewManager;
            swManipulator = (Manipulator)swModelViewMgr.CreateManipulator((int)swManipulatorType_e.swPlaneManipulator, swHdlr);
            swPlaneManipulator = (PlaneManipulator)swManipulator.GetSpecificManipulator();

            // Set the distance of plane
            swPlaneManipulator.Distance = 0.04;
            Debug.Print("Distance = " + swPlaneManipulator.Distance);

            //Set the angles of plane
            swPlaneManipulator.XAngle = 2 * PiVal() / 180;
            Debug.Print("X = " + swPlaneManipulator.XAngle);

            swPlaneManipulator.YAngle = 10 * PiVal() / 180;
            Debug.Print("Y = " + swPlaneManipulator.YAngle);

            // Set the height and width of plane
            swPlaneManipulator.Height = 0.1;
            Debug.Print("Height = " + swPlaneManipulator.Height);

            swPlaneManipulator.Width = 0.075;
            Debug.Print("Width = " + swPlaneManipulator.Width);
        }
    }
}
```

```

        // Set the color of plane to red
        swPlaneManipulator.Color = Information.RGB(255, 0, 0);

        // Update the plane's properties
        swPlaneManipulator.Update();

        // Show the plane with the manipulator
        swManipulator.Show(swModelDoc);
    }

    public double PiVal()
    {
        // Set PI
        return 4 * System.Math.Atan(1);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}

```

[Back to top](#)

//Class1.cs

```

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using System;
using System.Diagnostics;
using System.Runtime;

```

```

[System.Runtime.InteropServices.ComVisibleAttribute(true)]
public class Class1 : SwManipulatorHandler2
{

```

```

    public void SwManipulatorHandler2_OnUpdateDrag(object pManipulator, long handleIndex, object newPosMathPt)
    {
        Debug.Print("SwManipulatorHandler2_OnUpdateDrag");
        Debug.Print("    HandleIndex    = " + handleIndex);
        PlaneManipulator swRetManip = default(PlaneManipulator);
        swRetManip = (PlaneManipulator)pManipulator;

        if ((handleIndex == 8))
        {
            double retDist = 0;
            retDist = swRetManip.Distance;
        }
        else
        {
            double angleX = 0;
            double angleY = 0;
            angleX = swRetManip.XAngle;
            angleY = swRetManip.YAngle;
        }
    }

    public bool SwManipulatorHandler2_OnDelete(object pManipulator)
    {
        return false;
    }

    public void SwManipulatorHandler2_OnDirectionFlipped(object pManipulator)
    {
    }

    public bool SwManipulatorHandler2_OnDoubleValueChanged(object pManipulator, long Id, double Value)
    {
        return false;
    }

    public void SwManipulatorHandler2_OnEndNoDrag(object pManipulator, long handleIndex)
    {
    }

    public void SwManipulatorHandler2_OnHandleRmbSelected(object pManipulator, long handleIndex)
    {
    }

    public bool SwManipulatorHandler2_OnHandleLmbSelected(object pManipulator)
    {
        return false;
    }
}

```

```

public void SwManipulatorHandler2_OnHandleSelected(object pManipulator, long handleIndex)
{
}

public void SwManipulatorHandler2_OnItemSetFocus(object pManipulator, long Id)
{
}

public bool SwManipulatorHandler2_OnLmbSelected(object pManipulator)
{
    return false;
}

public bool SwManipulatorHandler2_OnStringValueChanged(object pManipulator, long Id, string Value)
{
    return false;
}

public void SwManipulatorHandler2_OnEndDrag(object pMani, long index)
{
}

public bool OnDelete(object pManipulator)
{
    return false;
}

public void OnDirectionFlipped(object pManipulator)
{
}

public bool OnDoubleValueChanged(object pManipulator, int handleIndex, ref double Value)
{
    return false;
}

public void OnEndDrag(object pManipulator, int handleIndex)
{
}

public void OnEndNoDrag(object pManipulator, int handleIndex)
{
}

public bool OnHandleLmbSelected(object pManipulator)
{
    return false;
}

public void OnHandleRmbSelected(object pManipulator, int handleIndex)
{
}

public void OnHandleSelected(object pManipulator, int handleIndex)
{
}

public void OnItemSetFocus(object pManipulator, int handleIndex)
{
}

public bool OnStringValueChanged(object pManipulator, int handleIndex, ref string Value)
{
    return false;
}

public void OnUpdateDrag(object pManipulator, int handleIndex, object newPosMathPt)
{
}
}

```

[Back to top](#)

Insert and Use Plane with Manipulator Example (VBA)

This example shows how to insert a plane that has a manipulator.

```

'-----
' Preconditions:
' 1. In the IDE:
'   a. Click Tools > References > SOLIDWORKS <version>
'      exposed type libraries for add-in.
'   b. Copy Module to the macro module.
'   c. Click Insert > Class Module and copy Class module
'      to that module.
'   d. Open the Immediate window.
' 2. Verify that the specified part exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Displays a plane with a manipulator.
' 3. Gets the distance, angles, height, and width of the plane.
' 4. Examine the Immediate window.
' 5. Hold down the left-mouse button and drag the
'    plane up and down using the manipulator, which calls the
'    the handler. Gets the handle index at each drag.
' 6. Click an edge of the plane and hold down the left-mouse button
'    and rotate the plane, which calls the handler. Gets the handle
'    index at each rotation.
' 7. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
'Module
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swModelViewMgr As SldWorks.ModelViewManager
Dim swHdlr As Class1
Dim swManipulator As SldWorks.Manipulator
Dim swPlaneManipulator As SldWorks.PlaneManipulator
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim PiVal As Double

Sub main()

    Set swHdlr = New Class1

    Set swApp = Application.SldWorks

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModelDoc = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

    ' Create a plane with a manipulator
    Set swModelViewMgr = swModelDoc.ModelViewManager
    Set swManipulator = swModelViewMgr.CreateManipulator(swManipulatorType_e.swPlaneManipulator, swHdlr)
    Set swPlaneManipulator = swManipulator.GetSpecificManipulator

    ' Set PI
    PiVal = 4 * Atn(1)

    ' Set the distance of the plane
    swPlaneManipulator.Distance = 0.04
    Debug.Print "Distance = " & swPlaneManipulator.Distance

    'Set the angles of the plane
    swPlaneManipulator.XAngle = 2 * PiVal / 180
    Debug.Print "X = " & swPlaneManipulator.XAngle

    swPlaneManipulator.YAngle = 10 * PiVal / 180
    Debug.Print "Y = " & swPlaneManipulator.YAngle

    ' Set the height and width of the plane
    swPlaneManipulator.Height = 0.1
    Debug.Print ("Height = " & swPlaneManipulator.Height)

    swPlaneManipulator.Width = 0.075
    Debug.Print ("Width = " & swPlaneManipulator.Width)

    ' Set the color of plane manipulator
    swPlaneManipulator.Color = RGB(255, 0, 0)

    ' Update the plane's properties
    swPlaneManipulator.Update

    ' Show plane manipulator
    swManipulator.Show swModelDoc

End Sub

```

End Sub

[Back to top](#)

' Class module

Option Explicit

Implements SwManipulatorHandler2

Private Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As Object)

```
    Debug.Print "SwManipulatorHandler2_OnUpdateDrag"
    Debug.Print "    HandleIndex      = " & handleIndex
    Dim swRetManip As SldWorks.PlaneManipulator
    Set swRetManip = pManipulator
```

```
    If (handleIndex = 8) Then
        Dim retDist As Double
        retDist = swRetManip.Distance
    Else
        Dim angleX As Double
        Dim angleY As Double
        angleX = swRetManip.XAngle
        angleY = swRetManip.YAngle
    End If
```

End Sub

Private Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean
End Function

Private Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object)
End Sub

Private Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As Double) As Boolean
End Function

Private Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
End Sub

Private Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
End Sub

Private Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean
End Function

Private Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
End Sub

Private Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Long)
End Sub

Private Function SwManipulatorHandler2_OnLmbSelected(ByVal pManipulator As Object) As Boolean
End Function

Private Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Long, Value As String) As Boolean
End Function

Private Sub SwManipulatorHandler2_OnEndDrag(ByVal pMani As Object, ByVal index As Long)
End Sub

[Back to top](#)

Insert and Use Plane with Manipulator Example (VB.NET)

This example shows how to insert a plane with a manipulator.

```

'-----
' Preconditions:
' 1. In the IDE:
'   a. Click Project > Add Reference > SolidWorks.Interop.swpublished
'      on the .NET tab.
'   b. Copy the code in SolidWorksMacro.vb to your project
'      and create a class and copy the code in Class1.vb
'      to that class.
'   c. Open the Immediate window.
' 2. Verify that the specified part exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Displays a plane with manipulator.
' 3. Gets the distance, angles, height, and width of the plane.
' 4. Examine the Immediate window.
' 5. Hold down the left-mouse button and drag the
'    plane up and down using the manipulator, which calls the
'    handler. Gets the handle index at each drag.
' 6. Click an edge of the plane and hold down the left-mouse button
'    and rotate the plane, which calls the handler. Gets the handle
'    index at each rotation.
' 7. Examine the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not
' save changes.
'-----

' SolidWorksMacro.vb
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports SolidWorks.Interop.swpublished

Partial Class SolidWorksMacro

    Public WithEvents swHdlr As Class1
    Public swManipulator As Manipulator
    Public swPlaneManipulator As PlaneManipulator

    Public Sub Main()

        Dim swModelDoc As ModelDoc2
        Dim swModelViewMgr As ModelViewManager

        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        swHdlr = New Class1

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
        swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)

        ' Create the plane with a manipulator
        swModelViewMgr = swModelDoc.ModelViewManager
        swManipulator = swModelViewMgr.CreateManipulator(swManipulatorType_e.swPlaneManipulator, swHdlr)
        swPlaneManipulator = swManipulator.GetSpecificManipulator

        ' Set the distance of plane
        swPlaneManipulator.Distance = 0.04
        Debug.Print("Distance = " & swPlaneManipulator.Distance)

        'Set the angles of plane
        swPlaneManipulator.XAngle = 2 * PiVal() / 180
        Debug.Print("X = " & swPlaneManipulator.XAngle)

        swPlaneManipulator.YAngle = 10 * PiVal() / 180
        Debug.Print("Y = " & swPlaneManipulator.YAngle)

        ' Set the height and width of plane
        swPlaneManipulator.Height = 0.1
        Debug.Print("Height = " & swPlaneManipulator.Height)

        swPlaneManipulator.Width = 0.075
        Debug.Print("Width = " & swPlaneManipulator.Width)

        ' Set the color of plane manipulator
        swPlaneManipulator.Color = RGB(255, 0, 0)

        ' Update the plane's properties
        swPlaneManipulator.Update()
    End Sub
End Class

```

```

        ' Show the plane with the manipulator
        swManipulator.Show(swModelDoc)

    End Sub

    Function PiVal() As Double
        ' Set PI
        PiVal = 4 * System.Math.Atan(1)
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>

    Public swApp As SldWorks

End Class

```

[Back to top](#)

```

'Class1.vb
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Diagnostics
Imports System.Runtime

<System.Runtime.InteropServices.ComVisibleAttribute(True)> _
Public Class Class1

    Implements SwManipulatorHandler2

    Public Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As
Object)
        'Public Sub SwManipulatorHandler2_OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Long, ByVal newPosMathPt As
Object) Implements SolidWorks.Interop.swpublished.SwManipulatorHandler.OnUpdateDrag
        Debug.Print("SwManipulatorHandler2_OnUpdateDrag")
        Debug.Print("    HandleIndex    = " & handleIndex)
        Dim swRetManip As PlaneManipulator
        swRetManip = pManipulator

        If (handleIndex = 8) Then
            Dim retDist As Double
            retDist = swRetManip.Distance
        Else
            Dim angleX As Double
            Dim angleY As Double
            angleX = swRetManip.XAngle
            angleY = swRetManip.YAngle
        End If
    End Sub

    Public Function SwManipulatorHandler2_OnDelete(ByVal pManipulator As Object) As Boolean
    End Function

    Public Sub SwManipulatorHandler2_OnDirectionFlipped(ByVal pManipulator As Object)
    End Sub

    Public Function SwManipulatorHandler2_OnDoubleValueChanged(ByVal pManipulator As Object, ByVal Id As Long, ByVal Value As Double)
As Boolean
    End Function

    Public Sub SwManipulatorHandler2_OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Long)
    End Sub

    Public Sub SwManipulatorHandler2_OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    End Sub

    Public Function SwManipulatorHandler2_OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean
    End Function

    Public Sub SwManipulatorHandler2_OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Long)
    End Sub

    Public Sub SwManipulatorHandler2_OnItemSetFocus(ByVal pManipulator As Object, ByVal Id As Long)
    End Sub

    Public Function SwManipulatorHandler2_OnLmbSelected(ByVal pManipulator As Object) As Boolean
    End Function

    Public Function SwManipulatorHandler2_OnStringValueChanged(ByVal pManipulator As Object, ByVal Id As Long, ByVal Value As String)
As Boolean
    End Function

    Public Sub SwManipulatorHandler2_OnEndDrag(ByVal pMani As Object, ByVal index As Long)
    End Sub

    Public Function OnDelete(ByVal pManipulator As Object) As Boolean Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnDelete
    End Function

```

```

    Public Sub OnDirectionFlipped(ByVal pManipulator As Object) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnDirectionFlipped
    End Sub

    Public Function OnDoubleValueChanged(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByRef Value As Double) As Boolean
Implements SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnDoubleValueChanged
    End Function

    Public Sub OnEndDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnEndDrag
    End Sub

    Public Sub OnEndNoDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnEndNoDrag
    End Sub

    Public Function OnHandleLmbSelected(ByVal pManipulator As Object) As Boolean Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnHandleLmbSelected
    End Function

    Public Sub OnHandleRmbSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnHandleRmbSelected
    End Sub

    Public Sub OnHandleSelected(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnHandleSelected
    End Sub

    Public Sub OnItemSetFocus(ByVal pManipulator As Object, ByVal handleIndex As Integer) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnItemSetFocus
    End Sub

    Public Function OnStringValueChanged(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByRef Value As String) As Boolean
Implements SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnStringValueChanged
    End Function

    Public Sub OnUpdateDrag(ByVal pManipulator As Object, ByVal handleIndex As Integer, ByVal newPosMathPt As Object) Implements
SolidWorks.Interop.swpublished.ISwManipulatorHandler2.OnUpdateDrag
    End Sub

End Class

```

[Back to top](#)

Insert Autoballoons Example (VBA)

This example shows how to insert autoballoons in a drawing document using the now obsolete method IDrawingDoc::AutoBalloon2.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Selects a drawing view.
' 3. Inserts autoballoons for each resolved component in
'    the selected drawing view.
' 4. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----

Sub main()

    Const swDetailingBalloonLayout_Default As Long = -1
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim vNoteArr As Variant
    Dim vNote As Variant
    Dim swNote As SldWorks.Note
    Dim swAnn As SldWorks.Annotation
    Dim vAttachPos As Variant
    Dim vAnnPos As Variant
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks

    ' Open drawing
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    bRet = swDraw.ActivateView(swView.Name): Debug.Assert bRet

    Debug.Print "File = " & swModel.GetPathName

    ' Get autoballoon settings
    Debug.Print "    DetailingBalloonsDisplayWithBentLeader = " & swModel.GetUserPreferenceToggle(swDetailingBalloonsDisplayWithBentLeader)
    Debug.Print "    DetailingBOMBalloonStyle = " & swModel.GetUserPreferenceIntegerValue(swDetailingBOMBalloonStyle)
    Debug.Print "    DetailingBOMBalloonFit = " & swModel.GetUserPreferenceIntegerValue(swDetailingBOMBalloonFit)
    Debug.Print "    DetailingAutoBalloonLayout = " & swModel.GetUserPreferenceIntegerValue(swDetailingAutoBalloonLayout)
    Debug.Print "    DetailingBalloonBentLeaderLength = " & swModel.GetUserPreferenceDoubleValue(swDetailingBalloonBentLeaderLength) * 1000# & " mm"

    Debug.Print ""
    Debug.Print "    " & swView.Name

    vNoteArr = swDraw.AutoBalloon2(swDetailingBalloonLayout_Default, False)

    ' Returns an empty array if:
    ' * Balloons already exist in any drawing view on any on sheet in
    '   the drawing document.
    ' * Drawing document is lightweight.
    ' Returns a note for each resolved component in the selected drawing view.
    If IsEmpty(vNoteArr) Then
        Debug.Print "    No balloons added."
        Exit Sub
    End If

    For Each vNote In vNoteArr
        Set swNote = vNote
        Set swAnn = swNote.GetAnnotation
        vAttachPos = swNote.GetAttachPos
        vAnnPos = swAnn.GetPosition

        Debug.Print "    Autoballoon text = " & swNote.GetText
        If swAnn.Layer = "" Then
            Debug.Print "        No layers defined."
        Else
            Debug.Print "        Layer = " & swAnn.Layer
        End If
        Debug.Print "    Note attachment point = (" & vAttachPos(0) * 1000# & ", " & vAttachPos(1) * 1000# & ", " & vAttachPos(2) * 1000# & ") mm"
        Debug.Print "    Origin of annotation = (" & vAnnPos(0) * 1000# & ", " & vAnnPos(1) * 1000# & ", " & vAnnPos(2) * 1000# & ") mm"
    Next

End Sub

```

Insert Autoballoons Example (VBA)

This example shows how to insert autoballoons in a drawing document using IDrawingDoc::AutoBalloon3.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Selects a drawing view.
' 3. Inserts autoballoons for each resolved component in
'    the selected drawing view.
' 4. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim vNoteArr As Variant
    Dim vNote As Variant
    Dim swNote As SldWorks.Note
    Dim swAnn As SldWorks.Annotation
    Dim vAttachPos As Variant
    Dim vAnnPos As Variant
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks

    ' Open drawing
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swDraw = swModel
    bRet = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    bRet = swDraw.ActivateView(swView.GetName2): Debug.Assert bRet

    Debug.Print "File = " & swModel.GetPathName

    Debug.Print " " & swView.GetName2

    ' Insert a split-circle autoballoon. Set the custom property
    ' for upper text to the model's custom property SW-Author
    ' and the model's lower text to the model's custom property SW-Comment,
    ' if both custom properties were set for the model

    vNoteArr = swDraw.AutoBalloon3(swDetailingBalloonLayout_Square, False, swBS_SplitCirc, swBF_5Chars, swBalloonTextCustomProperties, "SW-Author",
    swBalloonTextCustomProperties, "SW-Comments", "FORMAT")

    ' Returns an empty array if:
    ' * Balloons already exist in any drawing view on any on sheet in
    '   the drawing document.
    ' * Drawing document is lightweight.
    ' Returns a note for each resolved component in the selected drawing view.
    If IsEmpty(vNoteArr) Then
        Debug.Print "    No balloons added."
        Exit Sub
    End If

    For Each vNote In vNoteArr
        Set swNote = vNote
        Set swAnn = swNote.GetAnnotation
        vAttachPos = swNote.GetAttachPos
        vAnnPos = swAnn.GetPosition

        If swAnn.Layer = "" Then
            Debug.Print "    No layers defined."
        Else
            Debug.Print "    Layer = " & swAnn.Layer
        End If
        Debug.Print "    AttachPos = (" & vAttachPos(0) * 1000# & ", " & vAttachPos(1) * 1000# & ", " & vAttachPos(2) * 1000# & ") mm"
        Debug.Print "    AnnPos = (" & vAnnPos(0) * 1000# & ", " & vAnnPos(1) * 1000# & ", " & vAnnPos(2) * 1000# & ") mm"
    Next

End Sub

```

Insert Bend Table Example (C#)

This example shows how to insert a bend table in a drawing of a flattened sheet metal part.

```
//-----
// Preconditions:
// 1. Open a drawing that contains a flat pattern view
//    of a flattened sheet metal part.
// 2. Replace install_dir with your SOLIDWORKS installation directory.
// 3. Select the flat pattern view (Drawing View1)
//    in the FeatureManager design tree.
// 4. Ensure the namespace matches the name of your C# project.
//
// Postconditions:
// 1. A bend table is inserted for the selected view.
// 2. Inspect the Immediate Window.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertBendTable_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        View swView;
        BendTableAnnotation myBendTableAnnot;
        BendTable myBendTableFeat;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            swView = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObjectsDrawingView2(1, -1);
            myBendTableAnnot = swView.InsertBendTable(false, 0.3018189506239, 0.4876053587373, (int)
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "A", "install_dir\\lang\\english\\bendtable-standard.sldbndtblt");

            myBendTableFeat = myBendTableAnnot.BendTable;
            Debug.Print(myBendTableFeat.GetFeature().Name);
            Debug.Print("Starting tag: " + myBendTableFeat.StartingValue);
            Debug.Print("swBendTableTagStyle_e option: " + myBendTableFeat.TagStyle);
            Debug.Print("Number of bend table annotations: " + myBendTableFeat.GetTableAnnotationCount());

            Part.ClearSelection2(true);
        }

        public SldWorks swApp;
    }
}
```

Insert Bend Table Example (VBA)

This example shows how to insert a bend table in a drawing of a flattened sheet metal part.

```
'-----  
' Preconditions:  
' 1. Open a drawing that contains a flat pattern view  
'    of a flattened sheet metal part.  
' 2. Replace install_dir with your SOLIDWORKS installation directory.  
' 3. Select the flat pattern view (Drawing View1)  
'    in the FeatureManager design tree.  
'  
' Postconditions:  
' 1. A bend table is inserted for the selected view.  
' 2. Inspect the Immediate Window.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim swView As SldWorks.View  
Dim myBendTableAnnot As SldWorks.BendTableAnnotation  
Dim myBendTableFeat As SldWorks.BendTable  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
  
    Set swView = Part.SelectionManager.GetSelectedObjectsDrawingView2(1, -1)  
    Set myBendTableAnnot = swView.InsertBendTable(False, 0.3018189506239, 0.4876053587373, swBOMConfigurationAnchor_TopLeft, "A",  
"install_dir\lang\english\bendtable-standard.sldbndtbt")  
  
    Set myBendTableFeat = myBendTableAnnot.BendTable  
    Debug.Print myBendTableFeat.GetFeature.Name  
    Debug.Print "Starting tag: " & myBendTableFeat.StartingValue  
    Debug.Print "swBendTableTagStyle_e option: " & myBendTableFeat.TagStyle  
    Debug.Print "Number of bend table annotations: " & myBendTableFeat.GetTableAnnotationCount  
  
    Part.ClearSelection2 True  
  
End Sub
```

Insert Bend Table Example (VB.NET)

This example shows how to insert a bend table in a drawing of a flattened sheet metal part.

```
'-----
' Preconditions:
' 1. Open a drawing that contains a flat pattern view
'   of a flattened sheet metal part.
' 2. Replace install_dir with your SOLIDWORKS installation directory.
' 3. Select the flat pattern view (Drawing View1)
'   in the FeatureManager design tree.
'
' Postconditions:
' 1. A bend table is inserted for the selected view.
' 2. Inspect the Immediate Window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim swView As View
    Dim myBendTableAnnot As BendTableAnnotation
    Dim myBendTableFeat As BendTable
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        swView = Part.SelectionManager.GetSelectedObjectsDrawingView2(1, -1)
```

```
        myBendTableAnnot = swView.InsertBendTable(False, 0.3018189506239, 0.4876053587373,
```

```
        swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "A", "install_dir\lang\english\bendtable-standard.sldbndtbt")
```

```
        myBendTableFeat = myBendTableAnnot.BendTable
```

```
        Debug.Print(myBendTableFeat.GetFeature.Name)
```

```
        Debug.Print("Starting tag: " & myBendTableFeat.StartingValue)
```

```
        Debug.Print("swBendTableTagStyle_e option: " & myBendTableFeat.TagStyle)
```

```
        Debug.Print("Number of bend table annotations: " & myBendTableFeat.GetTableAnnotationCount)
```

```
        Part.ClearSelection2(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Insert Swept-cut Feature With Bidirectional Twist Example (VBA)

This example shows how to create a swept-cut feature with a bidirectional twist.

```

'-----
' Preconditions: Verify that the specified part exists.
'
' Postconditions:
' 1. Inserts a reference plane and circle sketch at a midpoint on
'   the sweep path of the model.
' 2. Creates a swept-cut feature with a bidirectional twist.
' 3. Examine the FeatureManager design tree and graphics area.
'
' Note: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swPathFeat As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swSweepFeatureData As SldWorks.SweepFeatureData
Dim swSweep As SldWorks.SweepFeatureData
Dim swProfileObj As Object
Dim swProfileBody As SldWorks.Body2
Dim swPathFeature As SldWorks.Feature
Dim myRefPlane As SldWorks.RefPlane
Dim skSegment As SldWorks.SketchSegment
Dim sketchLines As Variant
Dim status As Boolean
Dim boolstatus As Boolean
Dim longstatus As Long
Dim longwarnings As Long
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\sweepcutextrude2.SLDPRT", 1, 0, "", longstatus, longwarnings)
    swApp.ActivateDoc2 "sweepcutextrude.SLDPRT", False, longstatus
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    boolstatus = swModel.Extension.SelectByID2("Sketch4", "SKETCH", 3.68377611840314E-02, -1.01247230141019E-02, 0, False, 1,
Nothing, 0)
    boolstatus = swModel.Extension.SelectByID2("Sketch3", "SKETCH", -0.030956245593495, -0.018624786459474, 0, True, 4, Nothing, 0)

    Set swSweep = swModel.FeatureManager.CreateDefinition(swFmSweepCut)

    swSweep.Direction = 1 ' Bidirectional sweep
    swSweep.PathAlignmentType = 0
    swSweep.TwistControlType = 8
    swSweep.SetTwistAngle 31.12217 'Radians
    swSweep.D1ReverseTwistDir = False
    swSweep.SetD2TwistAngle 12.139 'Radians
    swSweep.D2ReverseTwistDir = True
    swSweep.TangentPropagation = False
    swSweep.AlignWithEndFaces = False
    swSweep.MaintainTangency = False
    swSweep.AdvancedSmoothing = False
    swSweep.StartTangencyType = 0
    swSweep.EndTangencyType = 0
    swSweep.ThinFeature = False
    swSweep.SetWallThickness True, 0#
    swSweep.SetWallThickness False, 0#
    swSweep.ThinWallType = 0
    swSweep.FeatureScope = True
    swSweep.AutoSelect = True
    swSweep.MergeSmoothFaces = True
    swSweep.AssemblyFeatureScope = True
    swSweep.AutoSelectComponents = True
    swSweep.PropagateFeatureToParts = False
    swSweep.CircularProfile = False
    swSweep.CircularProfileDiameter = 0#

    Set swPathFeat = swModel.FeatureManager.CreateFeature(swSweep)
End Sub

```

Insert BOM Balloon Example (VBA)

This example shows how to insert a BOM balloon.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Select an edge, silhouette, face, or vertex.  
'  
' Postconditions:  
' 1. Inserts a split-circle BOM balloon at  
'    the selected object.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelComp As SldWorks.Component2  
    Dim sCompName As String  
    Dim swNote As SldWorks.Note  
    Dim swAnn As SldWorks.Annotation  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swNote = swModel.InsertBOMBalloon2(swBS_SplitCirc, swBF_Tightest, swBalloonTextCustom, "Upper text", swBalloonTextCustom, "Lower text")  
  
End Sub
```

Insert BOM Table and BOM Balloon Example (C#)

This example shows how to insert a BOM table and a BOM balloon in a drawing document.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Inserts a parts-only BOM table.
// 2. Inserts a split-circle BOM balloon, which uses the BOM
//    table item number for its upper text,
//    to the selected edge.
// 3. Zoom to area and examine both the BOM table and BOM
//    balloon to verify.
//
// NOTE: Because this drawing document is used elsewhere,
// do not save any changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace InsertBOMBalloonModelDocExtension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            View swView = default(View);
            BomTableAnnotation swBOMAnnotation = default(BomTableAnnotation);
            BomFeature swBOMFeature = default(BomFeature);
            Note swNote = default(Note);
            bool boolstatus = false;
            int AnchorType = 0;
            int BomType = 0;
            string Configuration = null;
            string TableTemplate = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = swModel.Extension;
            boolstatus = swDrawing.ActivateView("Drawing View1");
            swView = (View)swDrawing.ActiveDrawingView;

            // Insert parts-only BOM table
            AnchorType = (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft;
            BomType = (int)swBomType_e.swBomType_PartsOnly;
            Configuration = "";
            TableTemplate = "";
            swBOMAnnotation = swView.InsertBomTable2(false, 0.4, 0.3, (int)AnchorType, (int)BomType, Configuration, TableTemplate);
            swBOMFeature = swBOMAnnotation.BomFeature;

            // Print the name of the configuration used for the BOM table
            Debug.Print("Name of configuration used for BOM table: " + swBOMFeature.Configuration);

            // Insert BOM balloon for the selected edge
            boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.1205506330468, 0.261655309417, -0.0004000000000133, false, 0,
            null, 0);
            swNote = (Note)swModelDocExt.InsertBOMBalloon((int)swBalloonStyle_e.swBS_SplitCirc, (int)swBalloonFit_e.swBF_Tightest,
            (int)swBalloonTextContent_e.swBalloonTextItemNumber, "", (int)swBalloonTextContent_e.swBalloonTextCustom, "Lower text",
            (int)swBalloonFit_e.swBF_UserDef, true, 2, "Denotation Text"
            );
            // Get whether balloon is a BOM balloon;
```

```
        // if so, print the name of the BOM balloon
        if (swNote.IsBomBalloon())
        {
            Debug.Print("Name of BOM balloon: " + swNote.GetName());
        }
    }
    public SldWorks swApp;
}
}
```

Insert BOM Table and BOM Balloon (VBA)

This example shows how to insert a BOM table and a BOM balloon in a drawing document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a parts-only BOM table.
' 2. Inserts a split-circle BOM balloon, which uses the BOM
'    table item number for its upper text,
'    to the selected edge.
' 3. Zoom to area and examine both the BOM table and BOM
'    balloon to verify.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save any changes.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModelDocExt        As SldWorks.ModelDocExtension
    Dim swDrawing            As SldWorks.DrawingDoc
    Dim swView               As SldWorks.View
    Dim swBOMAnnotation      As SldWorks.BomTableAnnotation
    Dim swBOMFeature         As SldWorks.BomFeature
    Dim swNote               As SldWorks.Note
    Dim boolstatus           As Boolean
    Dim AnchorType           As Long
    Dim BomType              As Long
    Dim Configuration        As String
    Dim TableTemplate        As String

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    boolstatus = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView

    ' Insert parts-only BOM table
    AnchorType = swBOMConfigurationAnchor_TopLeft
    BomType = swBomType_PartsOnly
    Configuration = ""
    TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    Set swBOMAnnotation = swView.InsertBomTable2(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate)
    Set swBOMFeature = swBOMAnnotation.BomFeature

    ' Print the name of the configuration used for the BOM table
    Debug.Print "Name of configuration used for BOM table: " & swBOMFeature.Configuration

    ' Insert BOM balloon for the selected edge
    boolstatus = swModelDocExt.SelectByID("", "EDGE", 0.1205506330468, 0.261655309417, -4.000000000133E-04, False, 0, Nothing, 0)
    Set swNote = swModelDocExt.InsertBOMBalloon(swBS_SplitCirc, swBF_Tightest, swBalloonTextItemNumber, "", swBalloonTextCustom,
"Lower text", swBF_UserDef, True, 2, "Denotation Text")

    ' Get whether balloon is a BOM balloon;
    ' if so, print the name of the BOM balloon
    If swNote.IsBomBalloon Then
        Debug.Print ("Name of BOM balloon: " & swNote.GetName)
    End If
```

End Sub

Insert BOM Table and BOM Balloon Example (VB.NET)

This example shows how to insert a BOM table and a BOM balloon in a drawing document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a parts-only BOM table.
' 2. Inserts a split-circle BOM balloon, which uses the BOM
'    table item number for its upper text,
'    to the selected edge.
' 3. Zoom to area and examine both the BOM table and BOM
'    balloon to verify.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save any changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swDrawing As DrawingDoc
```

```
        Dim swView As View
```

```
        Dim swBOMAnnotation As BomTableAnnotation
```

```
        Dim swBOMFeature As BomFeature
```

```
        Dim swNote As Note
```

```
        Dim boolstatus As Boolean
```

```
        Dim AnchorType As Integer
```

```
        Dim BomType As Integer
```

```
        Dim Configuration As String
```

```
        Dim TableTemplate As String
```

```
        swModel = swApp.ActiveDoc
```

```
        swDrawing = swModel
```

```
        swModelDocExt = swModel.Extension
```

```
        boolstatus = swDrawing.ActivateView("Drawing View1")
```

```
        swView = swDrawing.ActiveDrawingView
```

```
        ' Insert parts-only BOM table
```

```
        AnchorType = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft
```

```
        BomType = swBomType_e.swBomType_PartsOnly
```

```
        Configuration = ""
```

```
        TableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
```

```
        swBOMAnnotation = swView.InsertBomTable2(False, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate)
```

```
        swBOMFeature = swBOMAnnotation.BomFeature
```

```
        ' Print the name of the configuration used for the BOM table
```

```
        Debug.Print("Name of configuration used for BOM table: " & swBOMFeature.Configuration)
```

```
        ' Insert BOM balloon for the selected edge
```

```
        boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.1205506330468, 0.261655309417, -0.0004000000000133, False, 0, Nothing,
```

```
0)
```

```
        swNote = swModelDocExt.InsertBOMBalloon(swBalloonStyle_e.swBS_SplitCirc, swBalloonFit_e.swBF_Tightest,
swBalloonTextContent_e.swBalloonTextItemNumber, "", swBalloonTextContent_e.swBalloonTextCustom, "Lower text",
swBalloonFit_e.swBF_UserDef, True, 2, "Denotation Text")
```

```
' Get whether balloon is a BOM balloon;  
' if so, print the name of the BOM balloon  
If swNote.IsBomBalloon Then  
    Debug.Print("Name of BOM balloon: " & swNote.GetName)  
End If
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Insert BOM Table and Extract Data Example (C#)

This example shows how to insert a BOM table and extract data from it.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing document to open
// exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Selects a drawing view.
// 3. Inserts a BOM table at the point where the drawing
// was selected.
// 4. Examine the drawing and Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            View swView = default(View);
            BomTableAnnotation swBomTable = default(BomTableAnnotation);
            TableAnnotation swTable = default(TableAnnotation);
            double[] vPickPt = null;
            int nNumCol = 0;
            int nNumRow = 0;
            string sRowStr = null;
            string bomText = null;
            int len = 0;
            string subRowStr = null;
            int i = 0;
            int j = 0;
            bool bRet = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            //Open drawing document and select drawing view
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\driveworkspress\\mobile gantry.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swDrawing.ActivateView("Drawing View4");
            bRet = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.130207615492954, 0.11628112033195, 0, false, 0, null, 0);

            //Insert BOM table
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            vPickPt = (double[])swSelMgr.GetSelectionPoint2(1, -1);
            swBomTable = (BomTableAnnotation)swView.InsertBomTable2(false, vPickPt[0], vPickPt[1], (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, (int)swBomType_e.swBomType_Indented, "", "");
            swTable = (TableAnnotation)swBomTable;
            nNumCol = swTable.ColumnCount;
            nNumRow = swTable.RowCount;

            //List BOM contents
            for (i = 0; i <= nNumRow - 1; i++)
            {
                sRowStr = "";
                for (j = 0; j <= nNumCol - 1; j++)
                {
                    bomText = swTable.get_Text2(i, j, true);
                    sRowStr = sRowStr + bomText + ",";
                }
                len = sRowStr.Length - 1;
                subRowStr = sRowStr.Substring(0, len);
                Debug.Print(subRowStr);
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Insert BOM Table and Extract Data Example (VBA)

This example shows how to insert a BOM table and extract data from it.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Selects a drawing view.
' 3. Inserts a BOM table at the point where the drawing
'    was selected.
' 4. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim swBomTable As SldWorks.BomTableAnnotation
Dim swTable As SldWorks.TableAnnotation
Dim vPickPt As Variant
Dim nNumCol As Long
Dim nNumRow As Long
Dim sRowStr As String
Dim i As Long
Dim j As Long
Dim bRet As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open drawing document and select drawing view
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\driveworks\express\mobile gantry.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    bRet = swDrawing.ActivateView("Drawing View4")
    bRet = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.130207615492954, 0.11628112033195, 0, False, 0, Nothing, 0)

    'Insert BOM table
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
    Set swBomTable = swView.InsertBomTable2(False, vPickPt(0), vPickPt(1), swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft,
    swBomType_e.swBomType_Indented, "", "")
    Set swTable = swBomTable
    nNumCol = swTable.ColumnCount
    nNumRow = swTable.RowCount

    'List BOM contents
    For i = 0 To nNumRow - 1
        sRowStr = ""
        For j = 0 To nNumCol - 1
            sRowStr = sRowStr & swTable.Text2(i, j, True) & ", "
        Next j
        Debug.Print Left(sRowStr, Len(sRowStr) - 1)
    Next i

End Sub
```

Insert BOM Table and Extract Data Example (VB.NET)

This example shows how to insert a BOM table and extract data from it.

```

-----
Preconditions:
1. Verify that the specified drawing document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified drawing document.
2. Selects a drawing view.
3. Inserts a BOM table at the point where the drawing was selected.
4. Examine the drawing and Immediate window.
NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim swBomTable As BomTableAnnotation
        Dim swTable As TableAnnotation
        Dim vPickPt As Object
        Dim nNumCol As Integer
        Dim nNumRow As Integer
        Dim sRowStr As String
        Dim i As Integer
        Dim j As Integer
        Dim bRet As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        'Open drawing document and select drawing view
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\2018\samples\tutorial\driveworks\express\mobile gantry.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        swModelDocExt = swModel.Extension
        bRet = swDrawing.ActivateView("Drawing View4")
        bRet = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.130207615492954, 0.11628112033195, 0, False, 0, Nothing, 0)

        'Insert BOM table
        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1, -1)
        vPickPt = swSelMgr.GetSelectionPoint2(1, -1)
        swBomTable = swView.InsertBomTable2(False, vPickPt(0), vPickPt(1), swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, swBomType_e.swBomType_Indented, "", "")
        swTable = swBomTable
        nNumCol = swTable.ColumnCount
        nNumRow = swTable.RowCount

        'List BOM contents
        For i = 0 To nNumRow - 1
            sRowStr = ""
            For j = 0 To nNumCol - 1
                sRowStr = sRowStr & swTable.Text2(i, j, True) & ", "
            Next j
            Debug.Print(Left(sRowStr, Len(sRowStr) - 1))
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Insert BOM Table and Stacked Balloon Example (C#)

This example shows how to insert a bill of materials table and a stacked balloon annotation in an assembly document.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\flopress\ball_valve\ball_valve.sldasm.
//
// Postconditions:
// 1. Inserts a parts-only BOM table.
// 2. Inserts a stacked balloon annotation.
// 3. Inspect the Immediate Window for the name of the configuration used to create
//    the table and the name of the annotation.
//
// NOTE: Because this document is used do not save any changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace StackedBalloon_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            BomTableAnnotation swBOMAnnotation = default(BomTableAnnotation);
            BomFeature swBOMFeature = default(BomFeature);
            Note swNote = default(Note);
            StackedBalloonOptions StackedBalloonParams = default(StackedBalloonOptions);
            bool boolstatus = false;
            int BomType = 0;
            string Configuration = null;
            string TemplateName = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;

            // Insert BOM table
            TemplateName = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\lang\\english\\bom-standard.sldbomtbt";
            BomType = (int)swBomType_e.swBomType_PartsOnly;
            Configuration = "Default";
            swBOMAnnotation = swModelDocExt.InsertBomTable2(TemplateName, 40, 30, BomType, Configuration, false);
            swBOMFeature = swBOMAnnotation.BomFeature;

            // Print the name of the configuration used for the BOM table
            Debug.Print("Name of configuration used for BOM table: " + swBOMFeature.Configuration);

            // Select a face on which to attach the stacked balloons
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02268677135385, 0.0082159933431, 0.01133567172189, false, 0,
            null, 0);

            StackedBalloonParams = swModel.Extension.CreateStackedBalloonOptions();
            StackedBalloonParams.BalloonsPerLine = 10;
            StackedBalloonParams.StackDirection = (int)swStackedBalloonDirection_e.swStackedBalloonDir_Right;
            StackedBalloonParams.Style = (int)swBalloonStyle_e.swBS_Circular;
            StackedBalloonParams.Size = (int)swBalloonFit_e.swBF_5Chars;
            StackedBalloonParams.UpperTextContent = (int)swBalloonTextContent_e.swBalloonTextItemNumber;
            StackedBalloonParams.ShowQuantity = true;
            StackedBalloonParams.QuantityPlacement = (int)swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top;
            StackedBalloonParams.QuantityDenotationText = "PLACES";
            StackedBalloonParams.QuantityOverride = false;
            StackedBalloonParams.ItemNumberStart = 1;
            StackedBalloonParams.ItemNumberIncrement = 1;
            StackedBalloonParams.ItemOrder = (int)swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers;

            swNote = swModel.Extension.InsertStackedBalloon2(StackedBalloonParams);

            boolstatus = swModel.Extension.SelectByID2("", "FACE", -0.01632926202666, 0.05356671136803, 0.008058200827065, false,
            0, null, 0);
            boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.02199792044968, 0.00911087426303, 0.01062976811426, false, 0,
            null, 0);
            boolstatus = swModel.Extension.SelectByID2("", "FACE", -0.01412287126243, 0.003109265420704, -0.003984592306182, false,
            0, null, 0);

            swModel.ViewZoomtofit2();

            // Get whether balloon is a stacked balloon;
            // If so, print the name of the balloon

            if (swNote.IsStackedBalloon())
            {

```

```
        Debug.Print("Name of stacked balloons note: " + swNote.GetName());
    }
    swModel.ForceRebuild3(true);
}

public SldWorks swApp;
}
```

Insert BOM Table and Stacked Balloon Example (VBA)

This example shows how to insert a bill of materials table and a stacked balloon annotation in an assembly document.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\floypress\ball_valve\ball_valve.sldasm.
'
' Postconditions:
' 1. Inserts a parts-only BOM table.
' 2. Inserts a stacked balloon annotation.
' 3. Inspect the Immediate Window for the name of the configuration used to create
'    the table and the name of the annotation.
'
' NOTE: Because this document is used elsewhere, do not save any changes.
'-----

Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModelDocExt        As SldWorks.ModelDocExtension
    Dim swBOMAnnotation      As SldWorks.BomTableAnnotation
    Dim swBOMFeature         As SldWorks.BomFeature
    Dim swNote               As SldWorks.Note
    Dim StackedBalloonParams As SldWorks.StackedBalloonOptions
    Dim boolstatus           As Boolean
    Dim BomType              As Long
    Dim Configuration        As String
    Dim TemplateName         As String

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Insert BOM table
    TemplateName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    BomType = swBomType_PartsOnly
    Configuration = "Default"
    Set swBOMAnnotation = swModelDocExt.InsertBomTable2(TemplateName, 0.4, 0.3, BomType, Configuration, False)
    Set swBOMFeature = swBOMAnnotation.BomFeature

    ' Print the name of the configuration used for the BOM table
    Debug.Print "Name of configuration used for BOM table: " & swBOMFeature.Configuration

    ' Select a face on which to attach the stacked balloons
    boolstatus = swModelDocExt.SelectByID("", "FACE", -0.02268677135385, 0.0082159933431, 0.01133567172189, False, 0, Nothing, 0)

    Set StackedBalloonParams = swModel.Extension.CreateStackedBalloonOptions()
    StackedBalloonParams.BalloonsPerLine = 10
    StackedBalloonParams.StackDirection = swStackedBalloonDir_Right
    StackedBalloonParams.Style = swBS_Circular
    StackedBalloonParams.Size = swBF_5Chars
    StackedBalloonParams.UpperTextContent = swBalloonTextItemNumber
    StackedBalloonParams.ShowQuantity = True
    StackedBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_Top
    StackedBalloonParams.QuantityDenotationText = "PLACES"
    StackedBalloonParams.QuantityOverride = False
    StackedBalloonParams.ItemNumberStart = 1
    StackedBalloonParams.ItemNumberIncrement = 1
    StackedBalloonParams.ItemOrder = swBalloonItemNumbers_DoNotChangeItemNumbers

    Set swNote = swModel.Extension.InsertStackedBalloon2(StackedBalloonParams)

    boolstatus = swModel.Extension.SelectByID("", "FACE", -0.01632926202666, 0.05356671136803, 0.008058200827065, False, 0,
    Nothing, 0)
    boolstatus = swModel.Extension.SelectByID("", "FACE", 0.02199792044968, 0.00911087426303, 0.01062976811426, False, 0, Nothing,
    0)
    boolstatus = swModel.Extension.SelectByID("", "FACE", -0.01412287126243, 0.003109265420704, -0.003984592306182, False, 0,
    Nothing, 0)

    swModel.ViewZoomtofit2

    ' Get whether balloon is a stacked balloon;
    ' If so, print the name of the balloon
    If swNote.IsStackedBalloon Then
        Debug.Print ("Name of stacked balloons: note " & swNote.GetName)
    End If

    swModel.ForceRebuild3 True
End Sub

```

Insert BOM Table and Stacked Balloon Example (VB.NET)

This example shows how to insert a bill of materials table and a stacked balloon annotation in an assembly document.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\floxxpress\ball_valve\ball_valve.sldasm.
'-----
' Postconditions:
' 1. Inserts a parts-only BOM table.
' 2. Inserts a stacked balloon annotation.
' 3. Inspect the Immediate Window for the name of the configuration used to create
'    the table and the name of the annotation.
'-----
' NOTE: Because this document is used elsewhere, do not save any changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swBOMAnnotation As BomTableAnnotation
        Dim swBOMFeature As BomFeature
        Dim swNote As Note
        Dim StackedBalloonParams As StackedBalloonOptions
        Dim boolstatus As Boolean
        Dim BomType As Long
        Dim Configuration As String
        Dim TemplateName As String

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Insert BOM table
        TemplateName = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
        BomType = swBomType_e.swBomType_PartsOnly
        Configuration = "Default"
        swBOMAnnotation = swModelDocExt.InsertBomTable2(TemplateName, 0.4, 0.3, BomType, Configuration, False)
        swBOMFeature = swBOMAnnotation.BomFeature

        ' Print the name of the configuration used for the BOM table
        Debug.Print("Name of configuration used for BOM table: " & swBOMFeature.Configuration)

        ' Select a face on which to attach the stacked balloons
        boolstatus = swModelDocExt.SelectByID("", "FACE", -0.02268677135385, 0.0082159933431, 0.01133567172189, False, 0, Nothing,
0)

        StackedBalloonParams = swModel.Extension.CreateStackedBalloonOptions()
        StackedBalloonParams.BalloonsPerLine = 10
        StackedBalloonParams.StackDirection = swStackedBalloonDirection_e.swStackedBalloonDir_Right
        StackedBalloonParams.Style = swBalloonStyle_e.swBS_Circular
        StackedBalloonParams.Size = swBalloonFit_e.swBF_5Chars
        StackedBalloonParams.UpperTextContent = swBalloonTextContent_e.swBalloonTextItemNumber
        StackedBalloonParams.ShowQuantity = True
        StackedBalloonParams.QuantityPlacement = swBalloonQuantityPlacement_e.swBalloonQuantityPlacement_Top
        StackedBalloonParams.QuantityDenotationText = "PLACES"
        StackedBalloonParams.QuantityOverride = False
        StackedBalloonParams.ItemNumberStart = 1
        StackedBalloonParams.ItemNumberIncrement = 1
        StackedBalloonParams.ItemOrder = swBalloonItemNumbersOrder_e.swBalloonItemNumbers_DoNotChangeItemNumbers

        swNote = swModel.Extension.InsertStackedBalloon2(StackedBalloonParams)

        boolstatus = swModel.Extension.SelectByID("", "FACE", -0.01632926202666, 0.05356671136803, 0.008058200827065, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID("", "FACE", 0.02199792044968, 0.00911087426303, 0.01062976811426, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID("", "FACE", -0.01412287126243, 0.003109265420704, -0.003984592306182, False, 0,
Nothing, 0)

        swModel.ViewZoomtofit2()

        ' Get whether balloon is a stacked balloon;
        ' If so, print the name of the balloon

        If swNote.IsStackedBalloon Then
            Debug.Print("Name of stacked balloons note: " & swNote.GetName)
        End If

        swModel.ForceRebuild3(True)
    End Sub

```

Public swApp As SldWorks

End Class

Insert BOM Table Example (C#)

This example shows how to insert a BOM table using IView::InsertBomTable2.

```
//-----
// Preconditions: Open a drawing document and select
// a drawing view.
//
// Postconditions:
// 1. Inserts a BOM at the anchor point, if the
// drawing view does not already contain a BOM.
// 2. Examines the drawing and FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace SetConfigurationsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            FeatureManager swFeatMgr = default(FeatureManager);
            View swView = default(View);
            BomTableAnnotation swBomAnn = default(BomTableAnnotation);
            BomFeature swBomFeat = default(BomFeature);
            int AnchorType = 0;
            int BomType = 0;
            string Configuration = null;
            string TableTemplate = null;
            object Names = null;
            object Visible = null;
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeatMgr = (FeatureManager)swModel.FeatureManager;

            // Get selected drawing view
            swView = (View)swSelMgr.GetSelectedObject6(1, 0);
            AnchorType = (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft;
            BomType = (int)swBomType_e.swBomType_TopLevelOnly;
            Configuration = "";
            TableTemplate = "";

            // Insert BOM table
            swBomAnn = (BomTableAnnotation)swView.InsertBomTable2(true, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate);

            swModel.ClearSelection2(true);

            // Because BOM type is swBomType_TopLevelOnly,
            // then work with BomFeature to get and set configurations
            swBomFeat = (BomFeature)swBomAnn.BomFeature;
            Names = swBomFeat.GetConfigurations(false, ref Visible);
            Visible = true;
            boolstatus = swBomFeat.SetConfigurations(true, Visible, Names);

            // Update FeatureManager design tree
            swFeatMgr.UpdateFeatureTree();
        }
    }
    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Insert BOM Table Example (VBA)

This example shows how to insert a BOM table using IView::InsertBomTable2.

```
'-----  
' Preconditions: Open a drawing document and select  
' a drawing view.  
'  
' Postconditions:  
' 1. Inserts a BOM at the anchor point, if the  
' drawing view does not already contain a BOM.  
' 2. Examine the drawing and FeatureManager design tree.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeatMgr As SldWorks.FeatureManager  
Dim swView As SldWorks.View  
Dim swBomAnn As BomTableAnnotation  
Dim swBomFeat As SldWorks.BomFeature  
Dim AnchorType As Long  
Dim BomType As Long  
Dim Configuration As String  
Dim TableTemplate As String  
Dim Names As Variant  
Dim Visible As Variant  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeatMgr = swModel.FeatureManager  
  
    ' Get selected drawing view  
    Set swView = swSelMgr.GetSelectedObject6(1, 0)  
    AnchorType = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft  
    BomType = swBomType_e.swBomType_TopLevelOnly  
    Configuration = ""  
    TableTemplate = ""  
  
    ' Insert BOM table  
    Set swBomAnn = swView.InsertBomTable2(True, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate)  
  
    swModel.ClearSelection2 True  
  
    ' Because BOM type is swBomType_TopLevelOnly,  
    ' then work with BomFeature to get and set configurations  
    Set swBomFeat = swBomAnn.BomFeature  
    Names = swBomFeat.GetConfigurations(False, Visible)  
    Visible(0) = True  
    boolstatus = swBomFeat.SetConfigurations(True, Visible, Names)  
  
    ' Update FeatureManager design tree  
    swFeatMgr.UpdateFeatureTree  
  
End Sub
```

Insert BOM Table Example (VB.NET)

This example shows how to insert a BOM table using IView::InsertBomTable2.

```

-----
' Preconditions: Open a drawing document and select
' a drawing view.
' Postconditions:
' 1. Inserts a BOM at the anchor point, if the
'   drawing view does not already contain a BOM.
' 2. Examine the drawing and FeatureManager design tree.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

Partial Class SolidWorksMacro

```

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeatMgr As FeatureManager
        Dim swView As View
        Dim swBomAnn As BomTableAnnotation
        Dim swBomFeat As BomFeature
        Dim AnchorType As Integer
        Dim BomType As Integer
        Dim Configuration As String
        Dim TableTemplate As String
        Dim Names As Object
        Dim Visible As Object = Nothing
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeatMgr = swModel.FeatureManager

        ' Get selected drawing view
        swView = swSelMgr.GetSelectedObject6(1, 0)
        AnchorType = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft
        BomType = swBomType_e.swBomType_TopLevelOnly
        Configuration = ""
        TableTemplate = ""

        ' Insert BOM table
        swBomAnn = swView.InsertBomTable2(True, 0.4, 0.3, AnchorType, BomType, Configuration, TableTemplate)

        swModel.ClearSelection2(True)

        ' Because BOM type is swBomType_TopLevelOnly,
        ' then work with BomFeature to get and set configurations
        swBomFeat = swBomAnn.BomFeature
        Names = swBomFeat.GetConfigurations(False, Visible)
        Visible(0) = True
        boolstatus = swBomFeat.SetConfigurations(True, Visible, Names)

        ' Update FeatureManager design tree
        swFeatMgr.UpdateFeatureTree()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Insert Boundary Feature Example (C#)

This example shows how to insert and modify a boundary feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates two boss-extrude features.
// 3. Selects a face on each boss-extrude feature.
// 4. Creates a boundary feature using the the selected faces.
// 5. Gets and sets some boundary feature data.
// 6. Examine the Immediate window, FeatureManager design tree,
//    and the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            SketchManager swSketchMgr = default(SketchManager);
            BoundaryBossFeatureData swBoundaryBossFeatureData = default(BoundaryBossFeatureData);
            bool status = false;
            object[] sketchLines = null;
            int nbrCurves = 0;
            object directionVectorEntity = null;
            int directionVectorEntityType = 0;
            int tangencyType = 0;
            object[] d1Curves = null;
            int curveType = 0;
            int i = 0;

            //Open new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            //Create two boss-extrude features
            //and boundary feature
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDet
ailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.s
wDetailingNoOptionSpecified, true);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.107624731933299, 0.0371047716348016, 0, -0.083196263303762, -0.00987284730888405,
0);

            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.0508, 0.00254, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
0, 0, false);
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDet
ailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.s
wDetailingNoOptionSpecified, true);
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.0391822613366912, 0.0227443468893966, 0, 0.0479123594660678, -0.0893283426445919,
0);

            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeature = (Feature)swFeatureMgr.FeatureExtrusion3(true, false, false, 0, 0, 0.0508, 0.0508, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
0, 0, false);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0831962633037051, -0.000743092842242277, 0.0342529447572133, false, 8193, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0391822613366344, -0.00670639010576224, 0.0375699693011029, true, 16385, null, 0);
            swFeatureMgr.SetNetBlendCurveData(0, 0, 0, 1, true);
            swFeatureMgr.SetNetBlendCurveData(0, 1, 0, 0.26179938779915, 1, true);
            swFeatureMgr.SetNetBlendDirectionData(0, 32, 0, false, false);
            swFeatureMgr.SetNetBlendDirectionData(1, 32, 0, false, false);
            swFeatureMgr.InsertNetBlend(1, 2, 0, false, 0.0001, true, true, true, false,
-1, -1, false, -1, false, false, -1, false, -1, true);

            //Get boundary feature
            //Get and set some of its data
            status = swModelDocExt.SelectByID2("Boundary1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swBoundaryBossFeatureData = (BoundaryBossFeatureData)swFeature.GetDefinition();
            Debug.Print("Name of feature: " + swFeature.Name);
            swModel.ClearSelection2(true);
            status = swBoundaryBossFeatureData.AccessSelections(swModel, null);
            //Get number of curves
            nbrCurves = swBoundaryBossFeatureData.GetCurvesCount((int)swBoundaryBossDirection_e.swBoundaryBossDirection_First);
            Debug.Print(" Number of curves in Direction 1: " + nbrCurves);
            if (nbrCurves >= 0)
            {

```

```

//Get tangency type
tangencyType = swBoundaryBossFeatureData.GetGuideTangencyType((int)swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0);
if (tangencyType == (int)swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector)
{
    directionVectorEntity = swBoundaryBossFeatureData.GetDirectionVector((int)swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0);
    status = swSelectionMgr.AddSelectionListObject(directionVectorEntity, null);
    directionVectorEntityType = swSelectionMgr.GetSelectedObjectType3(1, -1);
    Debug.Print(" Type of entity selected for Direction Vector for curve 1 in Direction 1: " + directionVectorEntityType);
}
else
{
    Debug.Print(" Tangency type for curve 1 in Direction 1: " + tangencyType);
    Debug.Print(" NOTE: Tried to get entity for Direction Vector. Failed because");
    Debug.Print(" tangency type must be 2 (swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector);");
    Debug.Print(" so type of entity used for Direction Vector is not available.");
}
}
//Get and set draft angle
Debug.Print(" Original draft angle for curve 1 in Direction 1: " + swBoundaryBossFeatureData.GetDraftAngle((int)swBoundaryBossDirection_e.swB
oundaryBossDirection_First, 0));
swBoundaryBossFeatureData.SetDraftAngle((int)swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, 0.015);
Debug.Print(" Modified draft angle for curve 1 in Direction 1: " + swBoundaryBossFeatureData.GetDraftAngle((int)swBoundaryBossDirection_e.swB
oundaryBossDirection_First, 0));
//Flip draft angle
swBoundaryBossFeatureData.SetDraftAngleReverseDirection((int)swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, true);
Debug.Print(" Draft angle flipped for curve 1 in Direction 1: " + swBoundaryBossFeatureData.GetDraftAngleReverseDirection((int)swBoundaryBoss
Direction_e.swBoundaryBossDirection_First, 0));
//Get curves
swModel.ClearSelection2(true);
d1Curves = (object[])swBoundaryBossFeatureData.D1Curves;
for (i = 0; i < nbrCurves; i++)
{
    status = swSelectionMgr.AddSelectionListObject(d1Curves[i], null);
    curveType = swSelectionMgr.GetSelectedObjectType3(i + 1, -1);
    Debug.Print(" Curve " + (i + 1) + " type: " + curveType);
}
Debug.Print(" Is thin feature? " + swBoundaryBossFeatureData.IsThinFeature());
status = swFeature.ModifyDefinition(swBoundaryBossFeatureData, swModel, null);
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Insert Boundary Feature Example (VBA)

This example shows how to insert and modify a boundary feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates two boss-extrude features.
' 3. Selects a face on each boss-extrude feature.
' 4. Creates a boundary feature using the the selected faces.
' 5. Gets and sets some boundary feature data.
' 6. Examine the Immediate window, FeatureManager design tree,
'    and the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swSketchMgr As SldWorks.SketchManager
Dim swBoundaryBossFeatureData As SldWorks.BoundaryBossFeatureData
Dim status As Boolean
Dim sketchLines As Variant
Dim nbrCurves As Long
Dim directionVectorEntity As Object
Dim directionVectorEntityType As Long
Dim tangencyType As Long
Dim d1Curves As Variant
Dim curveType As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create two boss-extrude features
    'and boundary feature
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchMgr = swModel.SketchManager
    sketchLines = swSketchMgr.CreateCornerRectangle(-0.107624731933299, 3.71047716348016E-02, 0, -0.083196263303762, -9.87284730888405E-03, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, True, True, True, 0, 0, False)
    status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchLines = swSketchMgr.CreateCornerRectangle(-3.91822613366912E-02, 2.27443468893966E-02, 0, 4.79123594660678E-02, -8.93283426445919E-02, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.0508, 0.0508, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, True, True, True, 0, 0, False)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("", "FACE", -8.31962633037051E-02, -7.43092842242277E-04, 3.42529447572133E-02, False, 8193, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", -3.91822613366344E-02, -6.70639010576224E-03, 3.75699693011029E-02, True, 16385, Nothing, 0)
    swFeatureMgr.SetNetBlendCurveData 0, 0, 0, 0, 1, True
    swFeatureMgr.SetNetBlendCurveData 0, 1, 0, 0.26179938779915, 1, True
    swFeatureMgr.SetNetBlendDirectionData 0, 32, 0, False, False
    swFeatureMgr.SetNetBlendDirectionData 1, 32, 0, False, False
    swFeatureMgr.InsertNetBlend 1, 2, 0, False, 0.0001, True, True, True, True, True, False, -1, -1, False, -1, False, False, -1, False, -1, True

    'Get boundary feature
    'Get and set some of its data
    status = swModelDocExt.SelectByID2("Boundary1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swBoundaryBossFeatureData = swFeature.GetDefinition
    Debug.Print "Name of feature: " & swFeature.Name
    swModel.ClearSelection2 True
    status = swBoundaryBossFeatureData.AccessSelections(swModel, Nothing)
    'Get number of curves
    nbrCurves = swBoundaryBossFeatureData.GetCurvesCount(swBoundaryBossDirection_e.swBoundaryBossDirection_First)

```

```

Debug.Print " Number of curves in Direction 1: " & nbrCurves
If nbrCurves >= 0 Then
    'Get tangency type
    tangencyType = swBoundaryBossFeatureData.GetGuideTangencyType(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
    If tangencyType = swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector Then
        Set directionVectorEntity = swBoundaryBossFeatureData.GetDirectionVector(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
        status = swSelectionMgr.AddSelectionListObject(directionVectorEntity, Nothing)
        directionVectorEntityType = swSelectionMgr.GetSelectedObjectType3(1, -1)
        Debug.Print " Type of entity selected for Direction Vector for curve 1 in Direction 1: " & directionVectorEntityType
    Else
        Debug.Print " Tangency type for curve 1 in Direction 1: " & tangencyType
        Debug.Print " NOTE: Tried to get entity for Direction Vector. Failed because"
        Debug.Print " tangency type must be 2 (swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector),"
        Debug.Print " so type of entity used for Direction Vector is not available."
    End If
End If
'Get and set draft angle
Debug.Print " Original draft angle for curve 1 in Direction 1: " &
swBoundaryBossFeatureData.GetDraftAngle(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
swBoundaryBossFeatureData.SetDraftAngle swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, 0.015
Debug.Print " Modified draft angle for curve 1 in Direction 1: " &
swBoundaryBossFeatureData.GetDraftAngle(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
'Flip draft angle
swBoundaryBossFeatureData.SetDraftAngleReverseDirection swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, True
Debug.Print " Draft angle flipped for curve 1 in Direction 1: " &
swBoundaryBossFeatureData.GetDraftAngleReverseDirection(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
'Get curves
swModel.ClearSelection2 True
d1Curves = swBoundaryBossFeatureData.d1Curves
For i = 0 To nbrCurves - 1
    status = swSelectionMgr.AddSelectionListObject(d1Curves(i), Nothing)
    curveType = swSelectionMgr.GetSelectedObjectType3(i + 1, -1)
    Debug.Print " Curve " & i + 1 & " type: " & curveType
Next i
Debug.Print " Is thin feature? " & swBoundaryBossFeatureData.IsThinFeature
status = swFeature.ModifyDefinition(swBoundaryBossFeatureData, swModel, Nothing)

End Sub

```

Insert Boundary Feature Example (VB.NET)

This example shows how to insert and modify a boundary feature.

```

-----
Preconditions:
1. Verify that the specified part document template exists.
2. Open the Immediate window.

Postconditions:
1. Opens a new part.
2. Creates two boss-extrude features.
3. Selects a face on each boss-extrude feature.
4. Creates a boundary feature using the the selected faces.
5. Gets and sets some boundary feature data.
6. Examine the Immediate window, FeatureManager design tree,
   and the graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim swSketchMgr As SketchManager
        Dim swBoundaryBossFeatureData As BoundaryBossFeatureData
        Dim status As Boolean
        Dim sketchLines As Object
        Dim nbrCurves As Integer
        Dim directionVectorEntity As Object
        Dim directionVectorEntityType As Integer
        Dim tangencyType As Integer
        Dim d1Curves() As Object
        Dim curveType As Integer
        Dim i As Integer

        'Open new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Create two boss-extrude features
        'and boundary feature
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOption
Specified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, True)
        swSketchMgr = swModel.SketchManager
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.107624731933299, 0.0371047716348016, 0, -0.083196263303762, -0.00987284730888405, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.0508, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925
199433, False, False, False, False, True, True, True, 0, 0, False)
        status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOption
Specified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, True)
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.0391822613366912, 0.0227443468893966, 0, 0.0479123594660678, -0.0893283426445919, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeature = swFeatureMgr.FeatureExtrusion3(True, False, False, 0, 0, 0.0508, 0.0508, False, False, False, False, 0.0174532925199433, 0.01745329251
99433, False, False, False, False, True, True, True, 0, 0, False)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0831962633037051, -0.000743092842242277, 0.0342529447572133, False, 8193, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0391822613366344, -0.00670639010576224, 0.0375699693011029, True, 16385, Nothing, 0)
        swFeatureMgr.SetNetBlendCurveData(0, 0, 0, 1, True)
        swFeatureMgr.SetNetBlendCurveData(0, 1, 0, 0.26179938779915, 1, True)
        swFeatureMgr.SetNetBlendDirectionData(0, 32, 0, False, False)
        swFeatureMgr.SetNetBlendDirectionData(1, 32, 0, False, False)
        swFeatureMgr.InsertNetBlend(1, 2, 0, False, 0.0001, True, True, True, True, False, -1, -1, False, -1, False, False, -1, False, -1, True)

        'Get boundary feature
        'Get and set some of its data
        status = swModelDocExt.SelectByID2("Boundary1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swBoundaryBossFeatureData = swFeature.GetDefinition
        Debug.Print("Name of feature: " & swFeature.Name)
        swModel.ClearSelection2(True)
        status = swBoundaryBossFeatureData.AccessSelections(swModel, Nothing)
        'Get number of curves
        nbrCurves = swBoundaryBossFeatureData.GetCurvesCount(swBoundaryBossDirection_e.swBoundaryBossDirection_First)
        Debug.Print(" Number of curves in Direction 1: " & nbrCurves)
        If nbrCurves >= 0 Then
            'Get tangency type
            tangencyType = swBoundaryBossFeatureData.GetGuideTangencyType(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
            If tangencyType = swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector Then
                directionVectorEntity = swBoundaryBossFeatureData.GetDirectionVector(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0)
                status = swSelectionMgr.AddSelectionListObject(directionVectorEntity, Nothing)
                directionVectorEntityType = swSelectionMgr.GetSelectedObject3(1, -1)
                Debug.Print(" Type of entity selected for Direction Vector for curve 1 in Direction 1: " & directionVectorEntityType)
            Else
                Debug.Print(" Tangency type for curve 1 in Direction 1: " & tangencyType)
                Debug.Print(" NOTE: Tried to get entity for Direction Vector. Failed because")
            End If
        End If
    End Sub
End Class

```

```

        Debug.Print("    tangency type must be 2 (swBoundaryBossTangencyType_e.swBoundaryBossTangency_DirectionVector),")
        Debug.Print("    so type of entity used for Direction Vector is not available.")
    End If
End If
'Get and set draft angle
Debug.Print("    Original draft angle for curve 1 in Direction 1: " & swBoundaryBossFeatureData.GetDraftAngle(swBoundaryBossDirection_e.swBoundaryBo
ssDirection_First, 0))
swBoundaryBossFeatureData.SetDraftAngle(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, 0.015)
Debug.Print("    Modified draft angle for curve 1 in Direction 1: " & swBoundaryBossFeatureData.GetDraftAngle(swBoundaryBossDirection_e.swBoundaryBo
ssDirection_First, 0))
'Flip draft angle
swBoundaryBossFeatureData.SetDraftAngleReverseDirection(swBoundaryBossDirection_e.swBoundaryBossDirection_First, 0, True)
Debug.Print("    Draft angle flipped for curve 1 in Direction 1: " & swBoundaryBossFeatureData.GetDraftAngleReverseDirection(swBoundaryBossDirection
_e.swBoundaryBossDirection_First, 0))
'Get curves
swModel.ClearSelection2(True)
d1Curves = swBoundaryBossFeatureData.D1Curves
For i = 0 To nbrCurves - 1
    status = swSelectionMgr.AddSelectionListObject(d1Curves(i), Nothing)
    curveType = swSelectionMgr.GetSelectedObjectType3(i + 1, -1)
    Debug.Print("    Curve " & i + 1 & " type: " & curveType)
Next i
Debug.Print("    Is thin feature? " & swBoundaryBossFeatureData.IsThinFeature)
status = swFeature.ModifyDefinition(swBoundaryBossFeatureData, swModel, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Insert Boundary Surface Feature Example (VBA)

This example shows how to insert a boundary surface feature.

```
'-----
'Preconditions: Open install_dir\samples\tutorial\cosmosxpress\law_link.sldprt
'
'Postconditions:
'1. Inserts a boundary surface feature.
'2. Examine the FeatureManager design tree and graphics area.
'
'NOTE: Because the part is used elsewhere, do not save changes.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    swModel.ClearSelection2 True

    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    boolstatus = swModelDocExt.SelectByID2("", "EDGE", -9.640234260928E-04, 0.001963504230389, -0.006514073359085, False, 1, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("", "EDGE", -0.008533278872939, 0.001995620498424, -0.01250000018626, True, 2, Nothing, 0)

    Set swFeatMgr = swModel.FeatureManager

    swFeatMgr.SetNetBlendCurveData 0, 0, 0, 0, 1, True
    swFeatMgr.SetNetBlendDirectionData 0, 32, 0, False, False
    swFeatMgr.SetNetBlendCurveData 1, 0, 0, 0, 1, True
    swFeatMgr.SetNetBlendDirectionData 1, 32, 0, False, False
    swFeatMgr.InsertNetBlend 2, 1, 1, False, 0.0001, False, True, True, True, False, -1, -1, False, -1, False, False, -1, False, False, True

End Sub
```